Επεξηγηματικό Report 3ης Εργασίας

Ερώτημα A (a)-(c) και Ερώτημα B: Logistic Regression, LDA, QDA, Gaussian NB και KNN

ΑΕΜ: 6931

Απρίλιος 2026

Περιεχόμενα

1 Ποιο κομμάτι της εργασίας λύνουμε εδώ **4**
1.1 Η εκφώνηση, με απλά λόγια . . . 4
1.2 Τι είναι το Ερώτημα A . . . 4

2 Θεωρητικό υπόβαθρο **5**
2.1 Τι σημαίνει "πρόβλεψη τύπου κρασιού" . . . 5
2.2 Πώς κωδικοποιούμε τις δύο κατηγορίες . . . 6
2.3 Τι κάνει η Logistic Regression . . . 6
2.4 Πώς αποφασίζει τελικά red ή white . . . 6
2.5 Γιατί χωρίζουμε σε training και test . . . 7
2.6 Τι είναι ο confusion matrix . . . 7
2.7 Τα μέτρα αξιολόγησης . . . 7
2.7.1 Accuracy . . . 7
2.7.2 Sensitivity για white . . . 8
2.7.3 Specificity για red . . . 8
2.7.4 Balanced accuracy . . . 8
2.7.5 Precision και F1-score . . . 8
2.7.6 ROC καμπύλη και AUC . . . 9

3 Πώς λύσαμε το υποερώτημα A(a) **11**
3.1 Βήμα 1: Φόρτωση των δεδομένων . . . 11
3.2 Βήμα 2: Ορισμός στόχου . . . 11
3.3 Βήμα 3: Επιλογή μίας προβλεπτικής μεταβλητής κάθε φορά . . . 11
3.4 Βήμα 4: Διαχωρισμός σε training και test . . . 12
3.5 Βήμα 5: Εκπαίδευση Logistic Regression . . . 12
3.6 Βήμα 6: Πρόβλεψη και αξιολόγηση . . . 12

4 Αποτελέσματα Logistic Regression **13**
4.1 Συνοπτικός πίνακας . . . 13
4.2 Πείραμα 1: Logistic Regression με μεταβλητή alcohol . . . 14
4.2.1 Το μοντέλο που εκπαιδεύτηκε . . . 14
4.2.2 Confusion matrix . . . 14
4.2.3 Μέτρα αξιολόγησης . . . 14
4.2.4 Ερμηνεία με απλά λόγια . . . 15
4.3 Πείραμα 2: Logistic Regression με μεταβλητή pH . . . 15
4.3.1 Το μοντέλο που εκπαιδεύτηκε . . . 15
4.3.2 Confusion matrix . . . 15
4.3.3 Μέτρα αξιολόγησης . . . 16

4.3.4 Ερμηνεία με απλά λόγια . . . 16
4.4 Πείραμα 3: Logistic Regression με μεταβλητή chlorides . . . 16
4.4.1 Το μοντέλο που εκπαιδεύτηκε . . . 16
4.4.2 Confusion matrix . . . 17
4.4.3 Μέτρα αξιολόγησης . . . 17
4.4.4 Ερμηνεία με απλά λόγια . . . 18

5 Συμπέρασμα για το υποερώτημα A(a) 18

6 Υποερώτημα A(b): Linear Discriminant Analysis 19
6.1 Τι ζητάει το υποερώτημα A(b) . . . 19
6.2 Θεωρητικό υπόβαθρο: τι κάνει η LDA . . . 19
6.2.1 Η βασική υπόθεση της LDA . . . 20
6.2.2 Πώς αποφασίζει η LDA . . . 20
6.3 Πώς λύσαμε το υποερώτημα A(b) . . . 21
6.4 Κέντρα και διασπορά των δύο κλάσεων στο training set . . . 22
6.5 Συνοπτικά αποτελέσματα LDA . . . 23
6.6 Όρια απόφασης της LDA . . . 23
6.7 Πείραμα 1: LDA με μεταβλητή alcohol . . . 24
6.7.1 Το μοντέλο που εκπαιδεύτηκε . . . 24
6.7.2 Confusion matrix . . . 24
6.7.3 Μέτρα αξιολόγησης . . . 24
6.8 Πείραμα 2: LDA με μεταβλητή pH . . . 25
6.8.1 Το μοντέλο που εκπαιδεύτηκε . . . 25
6.8.2 Confusion matrix . . . 25
6.8.3 Μέτρα αξιολόγησης . . . 25
6.9 Πείραμα 3: LDA με μεταβλητή chlorides . . . 25
6.9.1 Το μοντέλο που εκπαιδεύτηκε . . . 25
6.9.2 Confusion matrix . . . 26
6.9.3 Μέτρα αξιολόγησης . . . 26
6.9.4 Γιατί βελτιώνεται η LDA με chlorides; . . . 26
6.10 Συμπέρασμα για το υποερώτημα A(b) . . . 27

7 Υποερώτημα A(c): Gaussian Naive Bayes 27
7.1 Τι ζητάει το υποερώτημα A(c) . . . 27
7.2 Θεωρητικό υπόβαθρο: η ιδέα του Bayes . . . 28
7.3 Γιατί λέγεται Gaussian Naive Bayes . . . 29
7.4 Πώς αποφασίζει το Gaussian NB . . . 29
7.5 Πώς λύσαμε το υποερώτημα A(c) . . . 30
7.6 Παράμετροι που έμαθε το Gaussian NB . . . 30
7.7 Συνοπτικά αποτελέσματα Gaussian NB . . . 30
7.8 Σχετικά όρια απόφασης . . . 31
7.9 Πείραμα 1: Gaussian NB με μεταβλητή alcohol . . . 31
7.9.1 Confusion matrix . . . 31
7.9.2 Μέτρα αξιολόγησης . . . 31
7.10 Πείραμα 2: Gaussian NB με μεταβλητή pH . . . 32
7.10.1 Confusion matrix . . . 32
7.10.2 Μέτρα αξιολόγησης . . . 32
7.11 Πείραμα 3: Gaussian NB με μεταβλητή chlorides . . . 32
7.11.1 Confusion matrix . . . 32
7.11.2 Μέτρα αξιολόγησης . . . 33
7.11.3 Γιατί το Gaussian NB με chlorides είναι καλύτερο από την LDA; . . . 33
7.12 Συμπέρασμα για το υποερώτημα A(c) . . . 33

8 Υποερώτημα A(ii): Καταλληλότερη προβλεπτική μεταβλητή **34**
8.1 Τι ζητάει το A(ii) . . . 34
8.2 Συνολική εικόνα ανά μεταβλητή . . . 34
8.3 Γιατί τα chlorides είναι καλύτερα; . . . 35
8.4 Απάντηση στο A(ii) . . . 35

9 A(iii): Σύγκριση με Chlorides **35**
9.1 Τι ζητάει το A(iii) . . . 35
9.2 Επίδοση στο test set . . . 35
9.3 Confusion matrices με chlorides . . . 36
9.4 Υπάρχει σημαντική διαφορά; . . . 36
9.5 Επιβεβαίωση με cross-validation . . . 36
9.6 Απάντηση στο A(iii) . . . 37

10 Ερώτημα B: Πρόβλεψη με όλες τις μεταβλητές **38**
10.1 Τι ζητάει το Ερώτημα B . . . 38
10.2 Γιατί βοηθάει η χρήση πολλών μεταβλητών . . . 38
10.3 Γιατί δεν ενώνουμε απλά τις μεταβλητές χωρίς σκέψη . . . 39
10.4 Τα δεδομένα για το Ερώτημα B . . . 39

11 Θεωρητικό υπόβαθρο για το Ερώτημα B **39**
11.1 Γενίκευση σε πολλές μεταβλητές . . . 40
11.2 K-Nearest Neighbors (KNN) . . . 40
11.2.1 Πώς μετράμε “κοντινό” . . . 40
11.2.2 Πώς ψηφίζουν οι γείτονες . . . 41
11.2.3 Γιατί το K είναι σημαντικό . . . 41
11.2.4 Τι είναι το 5-fold cross-validation . . . 41
11.3 Quadratic Discriminant Analysis (QDA) . . . 42
11.3.1 Πότε κερδίζει η QDA . . . 42
11.3.2 Πότε χάνει η QDA . . . 42
11.4 Τι δείχνει η ROC καμπύλη και η AUC σε αυτό το πλαίσιο . . . 42

12 Πώς λύσαμε το Ερώτημα B **42**
12.1 Δομή του κώδικα . . . 42
12.2 Τυποποίηση μέσω Pipeline . . . 43
12.3 Επιλογή του K με 5-fold CV . . . 43

13 Επιλογή του K με 5-fold CV **44**
13.1 Τι βρήκαμε . . . 44
13.2 Γιατί αυτό το K έχει νόημα . . . 44

14 Αποτελέσματα στο test set **45**
14.1 Συνοπτικός πίνακας . . . 45
14.2 Σύντομη υπενθύμιση των metrics . . . 45
14.3 Ερμηνεία των αποτελεσμάτων . . . 45
14.4 Γιατί η QDA είναι χειρότερη από τη LDA εδώ . . . 46
14.5 Confusion matrices . . . 46

15 ROC καμπύλες και AUC **46**
15.1 Η εικόνα . . . 46
15.2 Γιατί η AUC της Logistic Regression είναι η μεγαλύτερη, αλλά όχι η accuracy της . . . 47

16 Είναι στατιστικά σημαντικές οι διαφορές; 48
16.1 McNemar test 48
16.2 Τι μας λένε τα p-values 48

17 Τελικά συμπεράσματα για το Ερώτημα Β 49
17.1 Υπάρχει σημαντική διαφορά στην επίδοση των μεθόδων; 49
17.2 Ποια μέθοδος έχει την καλύτερη ακρίβεια; 49
17.3 Γιατί το Ερώτημα Β δίνει πολύ καλύτερα αποτελέσματα από το Ερώτημα Α 49
17.4 Σύνοψη 50

1 Ποιο κομμάτι της εργασίας λύνουμε εδώ

Η 3η εργασία μάς δίνει ένα σύνολο δεδομένων με κρασιά. Κάθε γραμμή του αρχείου αντιστοιχεί σε ένα κρασί και κάθε στήλη περιγράφει κάποιο χημικό χαρακτηριστικό του, όπως το `alcohol`, το `pH`, τα `chlorides` κ.λπ. Υπάρχει επίσης η στήλη `wine_type`, η οποία λέει αν το κρασί είναι `white` ή `red`.

1.1 Η εκφώνηση, με απλά λόγια

Η εκφώνηση ζητά να προβλέψουμε τον τύπο του κρασιού:

$$\text{wine_type} \in \{\text{red}, \text{white}\}.$$

Δηλαδή, θέλουμε να φτιάξουμε ένα μοντέλο που να κοιτάζει κάποια χημική μέτρηση ενός κρασιού και να απαντά:

"Πιστεύω ότι αυτό το κρασί είναι κόκκινο ή λευκό."

Πριν εκπαιδεύσουμε οποιοδήποτε μοντέλο, πρέπει να χωρίσουμε τα δεδομένα σε:

- **training set** 70%: τα δεδομένα από τα οποία το μοντέλο μαθαίνει.
- **test set** 30%: τα δεδομένα που κρατάμε στην άκρη για να ελέγξουμε αν το μοντέλο έμαθε κάτι χρήσιμο.

Ο χωρισμός γίνεται με `random_state = 6931`, δηλαδή με seed ίσο με τον ΑΕΜ. Αυτό σημαίνει ότι όποτε ξανατρέξουμε τον κώδικα, θα πάρουμε ακριβώς τον ίδιο διαχωρισμό σε training και test. Άρα τα αποτελέσματα είναι αναπαραγώγιμα.

1.2 Τι είναι το Ερώτημα Α

Το Ερώτημα Α ζητά να κάνουμε πρόβλεψη του τύπου κρασιού χρησιμοποιώντας **μία μόνο προβλεπτική μεταβλητή κάθε φορά**. Οι τρεις μεταβλητές που εξετάζονται είναι:

1. `alcohol`
2. `pH`
3. `chlorides`

Για κάθε μία από αυτές, η εργασία ζητά να εφαρμόσουμε τρεις μεθόδους ταξινόμησης:

1. Logistic Regression
2. Linear Discriminant Analysis
3. Gaussian Naive Bayes

Σε αυτό το report καλύπτουμε τα τρία πρώτα υποερωτήματα του Ερωτήματος Α: **A(a) Logistic Regression**, **A(b) Linear Discriminant Analysis** και **A(c) Gaussian Naive Bayes**. Άρα κάνουμε πρώτα τρία πειράματα με Logistic Regression:

1. Logistic Regression με μοναδική μεταβλητή το alcohol
2. Logistic Regression με μοναδική μεταβλητή το pH
3. Logistic Regression με μοναδική μεταβλητή τα chlorides

και έπειτα τα ίδια τρία πειράματα με LDA:

1. LDA με μοναδική μεταβλητή το alcohol
2. LDA με μοναδική μεταβλητή το pH
3. LDA με μοναδική μεταβλητή τα chlorides

και τέλος τα ίδια τρία πειράματα με Gaussian Naive Bayes:

1. Gaussian NB με μοναδική μεταβλητή το alcohol
2. Gaussian NB με μοναδική μεταβλητή το pH
3. Gaussian NB με μοναδική μεταβλητή τα chlorides

Για κάθε πείραμα υπολογίζουμε:

- τον **confusion matrix** στα test δεδομένα,
- την **accuracy**,
- την **balanced accuracy**,
- το **recall** για λευκά και κόκκινα κρασιά,
- το **precision**,
- το **F1-score**,
- και την **AUC**.

2 Θεωρητικό υπόβαθρο

2.1 Τι σημαίνει "πρόβλεψη τύπου κρασιού"

Στα δεδομένα μας η στήλη που θέλουμε να προβλέψουμε είναι η `wine_type`. Αυτή ονομάζεται **μεταβλητή στόχος** ή **target variable**. Οι στήλες που χρησιμοποιούμε για να τη μαντέψουμε ονομάζονται **προβλεπτικές μεταβλητές** ή **features**.

Εδώ η μεταβλητή στόχος έχει δύο δυνατές κατηγορίες:

$$\text{red}, \quad \text{white}.$$

Όταν θέλουμε να προβλέψουμε κατηγορία, δεν κάνουμε απλή αριθμητική πρόβλεψη. Κάνουμε **ταξινόμηση** (classification).

Παράδειγμα:

- Αν θέλαμε να προβλέψουμε την τιμή `alcohol = 11.2`, θα είχαμε πρόβλημα παλινδρόμησης.
- Αν θέλουμε να προβλέψουμε αν το κρασί είναι `red` ή `white`, έχουμε πρόβλημα ταξινόμησης.

Η Logistic Regression, παρότι έχει τη λέξη "Regression" στο όνομα, εδώ χρησιμοποιείται ως **μέθοδος ταξινόμησης**. Αυτό στην αρχή φαίνεται παράξενο, αλλά ο λόγος είναι ότι δεν προβλέπει κατευθείαν την κατηγορία. Πρώτα προβλέπει μια **πιθανότητα**.

2.2 Πώς κωδικοποιούμε τις δύο κατηγορίες

Οι αλγόριθμοι μηχανικής μάθησης δουλεύουν πιο εύκολα με αριθμούς παρά με λέξεις. Γι' αυτό μετατρέπουμε το `wine_type` ως εξής:

$$\text{red} = 0, \qquad \text{white} = 1.$$

Με αυτή τη σύμβαση, όταν το μοντέλο δίνει πιθανότητα, η πιθανότητα αυτή διαβάζεται ως:

$$P(\text{white} \mid x).$$

Δηλαδή:

> "Ποια είναι η πιθανότητα το κρασί να είναι λευκό, αν ξέρω την τιμή της μεταβλητής x;"

Το x εδώ είναι κάθε φορά μία από τις μεταβλητές:

$$x = \text{alcohol}, \quad x = \text{pH}, \quad x = \text{chlorides}.$$

2.3 Τι κάνει η Logistic Regression

Η Logistic Regression προσπαθεί να συνδέσει μία μεταβλητή x με την πιθανότητα ένα κρασί να είναι λευκό. Η σχέση δεν γράφεται απευθείας ως ευθεία πάνω στην πιθανότητα, γιατί οι πιθανότητες πρέπει πάντα να μένουν μεταξύ 0 και 1.

Αν γράφαμε κάτι σαν:

$$P(\text{white}) = \beta_0 + \beta_1 x,$$

τότε η δεξιά πλευρά θα μπορούσε να βγει μικρότερη από 0 ή μεγαλύτερη από 1. Αυτό δεν γίνεται να είναι πιθανότητα.

Η Logistic Regression λύνει αυτό το πρόβλημα με τη συνάρτηση sigmoid:

$$P(\text{white} \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}.$$

Η ποσότητα:

$$z = \beta_0 + \beta_1 x$$

μπορεί να πάρει οποιαδήποτε τιμή, από $-\infty$ μέχρι $+\infty$. Η sigmoid παίρνει αυτόν τον αριθμό και τον μετατρέπει σε πιθανότητα μεταξύ 0 και 1.

2.4 Πώς αποφασίζει τελικά red ή white

Αφού υπολογιστεί η πιθανότητα $P(\text{white} \mid x)$, χρειαζόμαστε έναν κανόνα απόφασης. Ο συνηθισμένος κανόνας είναι:

$$\begin{cases} \text{πρόβλεψη white}, & \text{αν } P(\text{white} \mid x) \geq 0.5, \\ \text{πρόβλεψη red}, & \text{αν } P(\text{white} \mid x) < 0.5. \end{cases}$$

Το 0.5 λέγεται **κατώφλι απόφασης** (decision threshold). Σημαίνει απλά:

> Αν το μοντέλο πιστεύει ότι η πιθανότητα για white είναι τουλάχιστον 50%, τότε προβλέπει white. Αλλιώς προβλέπει red.

Αυτό το σημείο είναι πολύ σημαντικό για την ερμηνεία των αποτελεσμάτων μας. Ένα μοντέλο μπορεί να κατατάσσει σχετικά καλά τα κρασιά από "πιο πιθανό λευκό" προς "πιο πιθανό κόκκινο", αλλά αν οι πιθανότητες σπάνια πέφτουν κάτω από 0.5, τότε με το default κατώφλι θα προβλέπει σχεδόν πάντα white.

2.5 Γιατί χωρίζουμε σε training και test

Αν εκπαιδεύαμε και αξιολογούσαμε το μοντέλο στα ίδια δεδομένα, θα ήταν σαν να βάζαμε σε έναν μαθητή ακριβώς τα ίδια θέματα που είχε δει στην προετοιμασία. Δεν θα ξέραμε αν πραγματικά κατάλαβε ή αν απλώς "θυμήθηκε" τα παραδείγματα.

Γι' αυτό κάνουμε:

$$70\% \text{ training}, \qquad 30\% \text{ test}.$$

Το μοντέλο βλέπει μόνο το training set. Το test set το βλέπει μόνο στο τέλος, όταν θέλουμε να μετρήσουμε την απόδοση.

Στη δική μας ανάλυση:

- συνολικά δεδομένα: 5.295 κρασιά,
- training set: 3.706 κρασιά,
- test set: 1.589 κρασιά.

Η κατανομή των κλάσεων στο συνολικό dataset είναι:

$$\text{white} : 3991\ (75{,}37\%), \qquad \text{red} : 1304\ (24{,}63\%).$$

Άρα το dataset είναι **ανισόρροπο**: υπάρχουν περίπου τριπλάσια λευκά κρασιά από κόκκινα. Αυτό έχει τεράστια σημασία, γιατί ένα πολύ απλοϊκό μοντέλο που προβλέπει πάντα `white` θα έχει ήδη accuracy περίπου 75%. Επομένως, η accuracy μόνη της μπορεί να μας ξεγελάσει.

2.6 Τι είναι ο confusion matrix

Ο **confusion matrix** είναι ένας πίνακας που δείχνει πόσες προβλέψεις ήταν σωστές και πόσες λάθος. Επειδή έχουμε δύο κλάσεις, ο πίνακας έχει 2 γραμμές και 2 στήλες.

Στο report χρησιμοποιούμε τη διάταξη:

	pred red	pred white
true red	TN	FP
true white	FN	TP

Οι γραμμές δείχνουν την πραγματική κατηγορία. Οι στήλες δείχνουν την πρόβλεψη του μοντέλου.

Με τη σύμβαση `white = 1`, η θετική κλάση είναι το white. Άρα:

- **TN** (True Negative): πραγματικά red και προβλέφθηκαν red.
- **FP** (False Positive): πραγματικά red αλλά προβλέφθηκαν white.
- **FN** (False Negative): πραγματικά white αλλά προβλέφθηκαν red.
- **TP** (True Positive): πραγματικά white και προβλέφθηκαν white.

2.7 Τα μέτρα αξιολόγησης

2.7.1 Accuracy

Η accuracy είναι το ποσοστό όλων των σωστών προβλέψεων:

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}.$$

Είναι εύκολο μέτρο, αλλά σε ανισόρροπα δεδομένα μπορεί να είναι παραπλανητικό. Στο δικό μας dataset, επειδή τα white είναι 75%, ένα μοντέλο που λέει πάντα white θα έχει περίπου 75% accuracy χωρίς να ξεχωρίζει καθόλου τα κόκκινα κρασιά.

2.7.2 Sensitivity για white

Το sensitivity για white, δηλαδή το recall της θετικής κλάσης, δείχνει:

Από όλα τα πραγματικά λευκά κρασιά, πόσα βρήκε σωστά το μοντέλο;

$$\text{Sensitivity}_{white} = \frac{TP}{TP + FN}.$$

2.7.3 Specificity για red

Το specificity για red δείχνει:

Από όλα τα πραγματικά κόκκινα κρασιά, πόσα βρήκε σωστά το μοντέλο;

$$\text{Specificity}_{red} = \frac{TN}{TN + FP}.$$

Αυτό το μέτρο είναι κρίσιμο εδώ, γιατί τα κόκκινα κρασιά είναι η μειοψηφική κλάση. Αν το specificity είναι 0, τότε το μοντέλο δεν αναγνωρίζει κανένα κόκκινο κρασί.

2.7.4 Balanced accuracy

Η balanced accuracy είναι ο μέσος όρος του recall για τις δύο κλάσεις:

$$\text{Balanced Accuracy} = \frac{\text{Sensitivity}_{white} + \text{Specificity}_{red}}{2}.$$

Αν ένα μοντέλο προβλέπει πάντα white:

$$\text{Sensitivity}_{white} = 1, \qquad \text{Specificity}_{red} = 0,$$

άρα:

$$\text{Balanced Accuracy} = \frac{1 + 0}{2} = 0.5.$$

Το 0.5 εδώ σημαίνει πρακτικά ότι το μοντέλο δεν κάνει ουσιαστική ταξινόμηση και λειτουργεί σαν ένας ταξινομητής που ευνοεί μόνο την πλειοψηφική κλάση.

2.7.5 Precision και F1-score

Το precision απαντά στην ερώτηση:

Όταν το μοντέλο προβλέπει μια κλάση, πόσο συχνά έχει δίκιο;

Για παράδειγμα, το precision για red δείχνει πόσα από τα κρασιά που προβλέφθηκαν ως red ήταν πράγματι red.

Το F1-score συνδυάζει precision και recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}.$$

Αν ένα μοντέλο δεν προβλέπει ποτέ red, τότε το F1 για red γίνεται 0.

2.7.6 ROC καμπύλη και AUC

Πριν εξηγήσουμε την AUC, πρέπει πρώτα να εξηγήσουμε τι είναι η **ROC καμπύλη**. Η ROC καμπύλη δεν είναι άλλη μία τελική πρόβλεψη του μοντέλου. Είναι ένας τρόπος να δούμε πώς συμπεριφέρεται το μοντέλο όταν αλλάζουμε το κατώφλι απόφασης.

Μέχρι τώρα χρησιμοποιούσαμε το συνηθισμένο κατώφλι 0.5:

$$\begin{cases} \text{white}, & \text{αν } P(\text{white}) \geq 0.5, \\ \text{red}, & \text{αν } P(\text{white}) < 0.5. \end{cases}$$

Όμως το 0.5 δεν είναι ο μόνος δυνατός κανόνας. Θα μπορούσαμε να είμαστε πιο αυστηροί και να πούμε:

$$\text{Πρόβλεψε white μόνο αν } P(\text{white}) \geq 0.8.$$

Ή θα μπορούσαμε να είμαστε πιο χαλαροί και να πούμε:

$$\text{Πρόβλεψε white αν } P(\text{white}) \geq 0.3.$$

Κάθε διαφορετικό κατώφλι αλλάζει τις τελικές προβλέψεις. Αν χαμηλώσουμε το κατώφλι, το μοντέλο προβλέπει πιο εύκολα white. Αν ανεβάσουμε το κατώφλι, το μοντέλο γίνεται πιο αυστηρό και προβλέπει λιγότερα white.

Η ROC καμπύλη δοκιμάζει πολλά τέτοια κατώφλια και για κάθε κατώφλι υπολογίζει δύο ποσότητες:

$$\text{True Positive Rate} = \frac{TP}{TP + FN}$$

και

$$\text{False Positive Rate} = \frac{FP}{FP + TN}.$$

Στο δικό μας πρόβλημα, επειδή έχουμε ορίσει `white = 1`, το **True Positive Rate** είναι το recall των white κρασιών:

> Από όλα τα πραγματικά white, πόσα αναγνωρίστηκαν ως white;

Το **False Positive Rate** δείχνει πόσα red κρασιά μπερδεύτηκαν ως white:

> Από όλα τα πραγματικά red, πόσα προβλέφθηκαν λανθασμένα ως white;

Άρα η ROC καμπύλη είναι ένα γράφημα με:

$$x\text{-άξονας} = \text{False Positive Rate}, \qquad y\text{-άξονας} = \text{True Positive Rate}.$$

Ιδανικά θέλουμε το μοντέλο να έχει υψηλό True Positive Rate και χαμηλό False Positive Rate. Δηλαδή θέλουμε να βρίσκει πολλά white χωρίς να μπερδεύει πολλά red ως white.

Πώς δημιουργείται βήμα-βήμα η ROC καμπύλη. Το μοντέλο Logistic Regression δεν δίνει μόνο την τελική κατηγορία `red` ή `white`. Για κάθε κρασί δίνει πρώτα μια πιθανότητα, π.χ.

$$P(\text{white}) = 0{,}91, \quad P(\text{white}) = 0{,}74, \quad P(\text{white}) = 0{,}42.$$

Η ROC καμπύλη φτιάχνεται σαν να βάζουμε όλα τα κρασιά σε μια σειρά, από αυτό που το μοντέλο θεωρεί πιο πιθανό να είναι white μέχρι αυτό που θεωρεί λιγότερο πιθανό να είναι white. Μετά μετακινούμε σταδιακά το κατώφλι από πολύ αυστηρό σε πολύ χαλαρό.

- Αν το κατώφλι είναι πάρα πολύ ψηλά, π.χ. πάνω από όλες τις πιθανότητες, τότε το μοντέλο δεν προβλέπει κανένα κρασί ως white. Άρα δεν βρίσκει κανένα white, αλλά δεν κάνει και κανένα red λάθος ως white. Το σημείο της ROC είναι:

$$(FPR, TPR) = (0, 0).$$

- Όσο χαμηλώνουμε το κατώφλι, αρχίζουν να προβλέπονται κάποια κρασιά ως white. Αν αυτά που προστίθενται είναι πράγματι white, τότε αυξάνεται το TPR, δηλαδή η καμπύλη ανεβαίνει προς τα πάνω.

- Αν όμως αυτά που προστίθενται είναι red που το μοντέλο τα μπερδεύει ως white, τότε αυξάνεται το FPR, δηλαδή η καμπύλη κινείται προς τα δεξιά.

- Αν το κατώφλι γίνει πάρα πολύ χαμηλό, τότε το μοντέλο προβλέπει όλα τα κρασιά ως white. Τότε έχει βρει όλα τα πραγματικά white, αλλά έχει μπερδέψει και όλα τα red ως white. Το τελευταίο σημείο είναι:

$$(FPR, TPR) = (1, 1).$$

Άρα η ROC καμπύλη ξεκινά πάντα από το κάτω αριστερό σημείο $(0, 0)$ και καταλήγει πάντα στο πάνω δεξί σημείο $(1, 1)$. Η σημαντική πληροφορία δεν είναι απλώς από πού ξεκινά και πού τελειώνει, αλλά **ποια διαδρομή** ακολουθεί ενδιάμεσα.

Γιατί παίρνει τη μορφή που παίρνει. Η μορφή της ROC καμπύλης δείχνει τη σειρά με την οποία το μοντέλο "μαζεύει" τα white και τα red καθώς χαμηλώνει το κατώφλι.

- Αν το μοντέλο είναι **καλό**, τότε δίνει υψηλές πιθανότητες white κυρίως στα πραγματικά white κρασιά. Άρα, στην αρχή που χαμηλώνουμε το κατώφλι, προστίθενται πολλά σωστά white και λίγα λάθος red. Η καμπύλη ανεβαίνει γρήγορα προς τα πάνω και μένει κοντά στην πάνω αριστερή γωνία.

- Αν το μοντέλο είναι **τυχαίο**, τότε τα white και τα red είναι ανακατεμένα στη σειρά πιθανοτήτων. Καθώς χαμηλώνουμε το κατώφλι, προστίθενται περίπου με τον ίδιο ρυθμό σωστά white και λάθος red. Τότε η ROC μοιάζει με διαγώνια γραμμή από το $(0, 0)$ στο $(1, 1)$.

- Αν το μοντέλο είναι **τέλειο**, τότε όλα τα πραγματικά white έχουν μεγαλύτερη πιθανότητα από όλα τα πραγματικά red. Η καμπύλη ανεβαίνει πρώτα κάθετα μέχρι το $(0, 1)$ και μετά πηγαίνει οριζόντια μέχρι το $(1, 1)$.

Στην πράξη, επειδή έχουμε πεπερασμένο αριθμό κρασιών στο test set, η ROC δεν είναι μια τέλεια λεία καμπύλη. Είναι περισσότερο μια κλιμακωτή γραμμή: κάθε φορά που το κατώφλι περνάει την πιθανότητα ενός συγκεκριμένου κρασιού, αλλάζει λίγο ο confusion matrix και άρα αλλάζει λίγο το TPR ή το FPR.

Σύνδεση με τις δικές μας μεταβλητές. Στη Logistic Regression με μία μεταβλητή, η μορφή της ROC εξαρτάται από το πόσο καλά χωρίζονται οι τιμές αυτής της μεταβλητής ανάμεσα στα red και white κρασιά.

Για το `alcohol`, οι τιμές των red και white επικαλύπτονται πολύ. Το μοντέλο δυσκολεύεται να βάλει πρώτα τα white και μετά τα red, οπότε η ROC είναι κοντά στη διαγώνιο και η AUC βγαίνει κοντά στο 0,5.

Για το `pH`, υπάρχει κάποια διαφορά ανάμεσα στις δύο κλάσεις, αλλά όχι πολύ καθαρή. Γι' αυτό η ROC είναι καλύτερη από τη διαγώνιο, αλλά όχι κοντά στην ιδανική πάνω αριστερή γωνία.

Για τα chlorides, οι τιμές ξεχωρίζουν πολύ καλύτερα τα δύο είδη κρασιού. Πολλά white παίρνουν υψηλότερη πιθανότητα white από πολλά red. Έτσι η ROC ανεβαίνει γρήγορα προς τα πάνω και η AUC γίνεται πολύ υψηλή.

Η **AUC** σημαίνει Area Under the Curve, δηλαδή **εμβαδό κάτω από την ROC καμπύλη**. Άρα, πιο σωστά:

- η **ROC** είναι η καμπύλη,
- η **AUC** είναι ένας αριθμός που συνοψίζει αυτή την καμπύλη.

Για να την καταλάβουμε διαισθητικά:

> Η AUC μετρά πόσο καλά το μοντέλο μπορεί να βάλει τα white και red κρασιά σε σωστή σειρά, από πιο πιθανό white προς λιγότερο πιθανό white.

Μια AUC κοντά στο 0.5 σημαίνει ότι το μοντέλο δεν ξεχωρίζει τις δύο κλάσεις. Μια AUC κοντά στο 1 σημαίνει πολύ καλή διακριτική ικανότητα.

Η AUC είναι χρήσιμη γιατί δεν εξαρτάται από το συγκεκριμένο κατώφλι 0.5. Μπορεί λοιπόν να είναι υψηλή ακόμη κι αν το default κατώφλι δεν δίνει ιδανικές τελικές προβλέψεις.

3 Πώς λύσαμε το υποερώτημα A(a)

3.1 Βήμα 1: Φόρτωση των δεδομένων

Φορτώσαμε το αρχείο:

`Data Analysis_2026 3rd Case_Data.csv`

το οποίο βρίσκεται στον φάκελο της 3ης εργασίας. Η εκφώνηση λέει ότι αυτά τα δεδομένα δεν χρειάζονται καθαρισμό, άρα δεν κάναμε διαδικασία διόρθωσης λαθών όπως στην 1η εργασία.

```
import pandas as pd

df = pd.read_csv("Data␣Analysis_2026␣3rd␣Case_Data.csv")
```

3.2 Βήμα 2: Ορισμός στόχου

Η μεταβλητή στόχος είναι η wine_type. Τη μετατρέψαμε σε αριθμητική:

```
y = (df["wine_type"] == "white").astype(int)
```

Αυτό σημαίνει:

$$\text{white} = 1, \qquad \text{red} = 0.$$

3.3 Βήμα 3: Επιλογή μίας προβλεπτικής μεταβλητής κάθε φορά

Για το υποερώτημα A(a), εκπαιδεύσαμε τρία ξεχωριστά μοντέλα Logistic Regression:

```
predictors = ["alcohol", "pH", "chlorides"]
```

Κάθε μοντέλο βλέπει μόνο μία στήλη. Για παράδειγμα, όταν εξετάζουμε το alcohol, το μοντέλο δεν βλέπει ούτε pH, ούτε chlorides, ούτε καμία άλλη χημική μεταβλητή.

Αυτό γίνεται επειδή το Ερώτημα A θέλει να δούμε πόσο χρήσιμη είναι κάθε μεταβλητή **μόνη της**.

3.4 Βήμα 4: Διαχωρισμός σε training και test

Χρησιμοποιήσαμε:

$$70\% \text{ training}, \qquad 30\% \text{ test},$$

με:

$$\texttt{random_state = 6931}.$$

Επιπλέον χρησιμοποιήθηκε `stratify=y`. Αυτό σημαίνει ότι ο χωρισμός κρατά περίπου την ίδια αναλογία red/white και στο training και στο test. Χωρίς stratification, θα μπορούσε τυχαία το test set να έχει λίγο διαφορετική σύνθεση κλάσεων, κάτι που θα έκανε την αξιολόγηση λιγότερο σταθερή.

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.30,
    random_state=6931,
    stratify=y
)
```

Με αυτόν τον χωρισμό προέκυψε:

Σύνολο	Πλήθος	red	white
Training set	3706	913	2793
Test set	1589	391	1198

3.5 Βήμα 5: Εκπαίδευση Logistic Regression

Για κάθε προβλεπτική μεταβλητή εκπαιδεύσαμε ένα μοντέλο:

```
from sklearn.linear_model import LogisticRegression

model = LogisticRegression(solver="lbfgs", max_iter=1000)
model.fit(X_train, y_train)
```

Το `fit` είναι το σημείο όπου το μοντέλο μαθαίνει τις παραμέτρους β_0 και β_1.

3.6 Βήμα 6: Πρόβλεψη και αξιολόγηση

Μετά την εκπαίδευση, ζητήσαμε από το μοντέλο να προβλέψει τις κατηγορίες των test δεδομένων:

```
y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
```

Το `y_pred` περιέχει τις τελικές προβλέψεις 0 ή 1. Το `y_proba` περιέχει τις πιθανότητες για white.

Έπειτα υπολογίσαμε τον confusion matrix και τα μέτρα αξιολόγησης:

```
from sklearn.metrics import (
    confusion_matrix,
    accuracy_score,
    balanced_accuracy_score,
    roc_auc_score,
    f1_score
)

cm = confusion_matrix(y_test, y_pred)
```

```
accuracy = accuracy_score(y_test, y_pred)
balanced_accuracy = balanced_accuracy_score(y_test, y_pred)
auc = roc_auc_score(y_test, y_proba)
```

4 Αποτελέσματα Logistic Regression

4.1 Συνοπτικός πίνακας

Μεταβλητή	Accuracy	Bal. Acc.	AUC	Recall white	Recall red	F1 red
alcohol	0,7539	0,5000	0,5219	1,0000	0,0000	0,0000
pH	0,7577	0,5335	0,6798	0,9750	0,0921	0,1575
chlorides	0,7558	0,5236	0,9220	0,9808	0,0665	0,1182

Με μια πρώτη ματιά, οι τρεις accuracies φαίνονται πολύ κοντά:

$$0{,}7539, \quad 0{,}7577, \quad 0{,}7558.$$

Αν όμως κοιτούσαμε μόνο αυτό, θα βγάζαμε λάθος συμπέρασμα. Πρέπει να θυμόμαστε ότι το test set έχει 1198 white και 391 red. Άρα ένα μοντέλο που λέει πάντα white έχει ήδη:

$$\frac{1198}{1589} = 0{,}7539.$$

Γι' αυτό εξετάζουμε και balanced accuracy, recall red και AUC.
Για να διαβάσουμε σωστά τον πίνακα, θυμόμαστε τι μας λέει κάθε metric:

- **Accuracy**: μετρά το συνολικό ποσοστό σωστών προβλέψεων. Στα δικά μας αποτελέσματα είναι περίπου 75% και για τις τρεις μεταβλητές. Αυτό όμως δεν σημαίνει απαραίτητα ότι τα μοντέλα είναι καλά, γιατί περίπου 75% των test δεδομένων είναι ήδη `white`.

- **Balanced Accuracy**: μετρά την απόδοση πιο δίκαια, επειδή λαμβάνει υπόψη και το πόσο καλά βρίσκουμε τα white και το πόσο καλά βρίσκουμε τα red. Εδώ οι τιμές είναι κοντά στο 0,5, άρα τα μοντέλα δεν έχουν ισορροπημένη επίδοση στις δύο κλάσεις.

- **AUC**: μετρά πόσο καλά το μοντέλο βάζει τα κρασιά σε σειρά από πιο πιθανό white προς λιγότερο πιθανό white, ανεξάρτητα από το συγκεκριμένο threshold 0.5. Εδώ φαίνεται η μεγάλη διαφορά των `chlorides`: ενώ η accuracy τους δεν είναι εντυπωσιακή, η AUC είναι 0,9220, άρα η μεταβλητή έχει πολύ ισχυρή διακριτική πληροφορία. Αυτό σημαίνει ότι τα `chlorides` βάζουν τα κρασιά σε πολύ σωστή σειρά, ακόμη κι αν το τελικό κατώφλι 0.5 δεν μετατρέπει αυτή τη σωστή σειρά σε πολλές σωστές προβλέψεις red.

- **Recall white**: δείχνει από όλα τα πραγματικά white κρασιά πόσα βρήκε σωστά το μοντέλο. Είναι πολύ υψηλό και στις τρεις περιπτώσεις, επειδή τα μοντέλα τείνουν να προβλέπουν συχνά `white`.

- **Recall red**: δείχνει από όλα τα πραγματικά red κρασιά πόσα βρήκε σωστά το μοντέλο. Αυτό είναι το αδύναμο σημείο μας: το `alcohol` έχει recall red 0, το `pH` 0,0921 και τα `chlorides` 0,0665.

- **F1 red**: συνδυάζει το precision και το recall για την κλάση red. Επειδή το recall red είναι πολύ χαμηλό, το F1 red είναι επίσης χαμηλό. Αυτό δείχνει ότι, με το default threshold, η Logistic Regression δυσκολεύεται να λειτουργήσει καλά για τη μειοψηφική κλάση.

Άρα το βασικό συμπέρασμα του συνοπτικού πίνακα είναι το εξής: η accuracy μάς δίνει μια πρώτη εικόνα, αλλά μπορεί να μας παραπλανήσει λόγω της ανισορροπίας των κλάσεων. Το recall red και το F1 red δείχνουν ότι τα κόκκινα κρασιά δεν αναγνωρίζονται καλά, ενώ η AUC δείχνει ότι τα `chlorides` είναι η μεταβλητή με την καλύτερη πραγματική διακριτική ικανότητα.

4.2 Πείραμα 1: Logistic Regression με μεταβλητή alcohol

4.2.1 Το μοντέλο που εκπαιδεύτηκε

Για τη μεταβλητή `alcohol`, το μοντέλο έμαθε:

$$P(\text{white} \mid alcohol) = \frac{1}{1 + e^{-(0{,}009928 + 0{,}105625 \cdot alcohol)}}.$$

Ο συντελεστής του `alcohol` είναι θετικός:

$$\beta_1 = 0{,}105625.$$

Αυτό σημαίνει ότι, σύμφωνα με το μοντέλο, όσο αυξάνεται το alcohol, αυξάνεται λίγο και η πιθανότητα για white. Όμως το "λίγο" εδώ είναι πολύ σημαντικό. Ο συντελεστής είναι μικρός και η μεταβλητή δεν ξεχωρίζει ουσιαστικά τις δύο κλάσεις.

4.2.2 Confusion matrix

	Predicted red	Predicted white
True red	0	391
True white	0	1198

Αυτός ο πίνακας μάς λέει κάτι πολύ καθαρό:

- Από τα 391 πραγματικά κόκκινα κρασιά, το μοντέλο δεν βρήκε κανένα. Όλα τα είπε white.
- Από τα 1198 πραγματικά λευκά κρασιά, τα βρήκε όλα σωστά, επειδή είπε white σε κάθε περίπτωση.

Άρα το μοντέλο δεν έκανε ουσιαστική ταξινόμηση. Απλώς προέβλεπε πάντα την πλειοψηφική κλάση.

4.2.3 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7539
Balanced Accuracy	0,5000
AUC	0,5219
Recall white	1,0000
Recall red	0,0000
Precision white	0,7539
Precision red	0,0000
F1 white	0,8597
F1 red	0,0000

Η accuracy είναι 75,39%, αλλά αυτό δεν είναι επιτυχία. Είναι ακριβώς το ποσοστό των white στο test set. Η balanced accuracy είναι 0,5, δηλαδή το μοντέλο δεν έχει πραγματική χρησιμότητα ως ταξινομητής δύο κλάσεων.

Η AUC είναι 0,5219, πολύ κοντά στο 0,5. Αυτό σημαίνει ότι ακόμη και ως βαθμολόγηση πιθανοτήτων, το `alcohol` δεν βοηθά αρκετά για να ξεχωρίσουμε red από white.

4.2.4 Ερμηνεία με απλά λόγια

Το alcohol μόνο του δεν αρκεί. Τα κόκκινα και τα λευκά κρασιά στο dataset έχουν τόσο κοντινές τιμές alcohol, ώστε το μοντέλο δεν βρίσκει καθαρό όριο που να τα χωρίζει.

Οι ενδεικτικές πιθανότητες για white που δίνει το μοντέλο είναι:

alcohol	9,2	9,6	10,4	11,2	12,2
$P(\text{white})$	0,7274	0,7357	0,7518	0,7673	0,7856

Όλες αυτές οι πιθανότητες είναι πάνω από 0,5. Άρα το μοντέλο προβλέπει white σχεδόν παντού.

4.3 Πείραμα 2: Logistic Regression με μεταβλητή pH

4.3.1 Το μοντέλο που εκπαιδεύτηκε

Για τη μεταβλητή `pH`, το μοντέλο έμαθε:

$$P(\text{white} \mid pH) = \frac{1}{1 + e^{-(14{,}749484 - 4{,}203535 \cdot pH)}}.$$

Ο συντελεστής του `pH` είναι αρνητικός:

$$\beta_1 = -4{,}203535.$$

Αυτό σημαίνει ότι όσο αυξάνεται το pH, η πιθανότητα για white μειώνεται. Με άλλα λόγια, υψηλότερες τιμές pH κάνουν το μοντέλο να γέρνει περισσότερο προς red.

4.3.2 Confusion matrix

	Predicted red	Predicted white
True red	36	355
True white	30	1168

Ας το διαβάσουμε προσεκτικά:

- Υπήρχαν 391 πραγματικά red στο test set.
- Από αυτά, το μοντέλο βρήκε σωστά 36.
- Τα υπόλοιπα 355 red τα μπέρδεψε και τα είπε white.
- Υπήρχαν 1198 πραγματικά white.
- Από αυτά, βρήκε σωστά 1168.
- Μόνο 30 white τα είπε λανθασμένα red.

Σε σχέση με το alcohol, εδώ το μοντέλο αρχίζει να αναγνωρίζει μερικά κόκκινα κρασιά. Όμως εξακολουθεί να χάνει τα περισσότερα.

4.3.3 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7577
Balanced Accuracy	0,5335
AUC	0,6798
Recall white	0,9750
Recall red	0,0921
Precision white	0,7669
Precision red	0,5455
F1 white	0,8585
F1 red	0,1575

Η accuracy είναι 0,7577, δηλαδή λίγο μεγαλύτερη από το απλό "λέω πάντα white". Όμως η balanced accuracy είναι μόνο 0,5335. Αυτό σημαίνει ότι η βελτίωση είναι μικρή.

Το recall red είναι:

$$0{,}0921.$$

Δηλαδή το μοντέλο βρίσκει μόνο περίπου 9,21% των κόκκινων κρασιών.

Η AUC είναι 0,6798. Αυτό είναι καλύτερο από το alcohol και δείχνει ότι το pH έχει κάποια διακριτική πληροφορία, αλλά όχι αρκετή για πολύ καλή ταξινόμηση με μία μόνο μεταβλητή.

4.3.4 Ερμηνεία με απλά λόγια

Το pH βοηθά περισσότερο από το alcohol. Το μοντέλο καταλαβαίνει ότι υψηλότερο pH συνδέεται περισσότερο με red. Όμως οι δύο κλάσεις εξακολουθούν να επικαλύπτονται πολύ.

Ενδεικτικά:

pH	3,04	3,13	3,21	3,31	3,41
$P(\text{white})$	0,8777	0,8310	0,7784	0,6976	0,6024

Βλέπουμε ότι όσο το pH αυξάνεται, η πιθανότητα για white πέφτει. Παρ' όλα αυτά, ακόμη και στο pH 3,41 η πιθανότητα παραμένει πάνω από 0,5. Άρα το μοντέλο εξακολουθεί να προβλέπει white σε πολλές περιπτώσεις.

4.4 Πείραμα 3: Logistic Regression με μεταβλητή chlorides

4.4.1 Το μοντέλο που εκπαιδεύτηκε

Για τη μεταβλητή `chlorides`, το μοντέλο έμαθε:

$$P(\text{white} \mid chlorides) = \frac{1}{1 + e^{-(1{,}978076 - 14{,}922372 \cdot chlorides)}}.$$

Ο συντελεστής είναι αρνητικός:

$$\beta_1 = -14{,}922372.$$

Άρα όσο αυξάνονται τα chlorides, τόσο μειώνεται η πιθανότητα για white και τόσο αυξάνεται η ένδειξη προς red.

Το μέγεθος του συντελεστή είναι μεγάλο, αλλά πρέπει να προσέξουμε ότι τα `chlorides` έχουν μικρές αριθμητικές τιμές, π.χ. 0,03 ή 0,08. Επομένως δεν συγκρίνουμε απευθείας το $-14{,}92$ με τους άλλους συντελεστές χωρίς να λάβουμε υπόψη την κλίμακα της μεταβλητής.

4.4.2 Confusion matrix

	Predicted red	Predicted white
True red	26	365
True white	23	1175

Η ανάγνωση είναι:

- Από τα 391 πραγματικά red, το μοντέλο βρήκε σωστά 26.
- Τα 365 red τα είπε λανθασμένα white.
- Από τα 1198 πραγματικά white, βρήκε σωστά 1175.
- Μόνο 23 white τα είπε λανθασμένα red.

Με το default κατώφλι 0,5, το μοντέλο παραμένει πολύ συντηρητικό στο να προβλέψει red. Προτιμά να πει white εκτός αν τα chlorides είναι αρκετά υψηλά.

4.4.3 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7558
Balanced Accuracy	0,5236
AUC	0,9220
Recall white	0,9808
Recall red	0,0665
Precision white	0,7630
Precision red	0,5306
F1 white	0,8583
F1 red	0,1182

Εδώ υπάρχει ένα πολύ ενδιαφέρον σημείο. Η accuracy και η balanced accuracy δεν φαίνονται εντυπωσιακές. Όμως η AUC είναι:

$$0{,}9220.$$

Αυτό είναι πολύ υψηλό και στην αρχή φαίνεται αντιφατικό: πώς γίνεται η accuracy να είναι μόνο 0,7558, αλλά η AUC να είναι 0,9220; Η απάντηση είναι ότι τα δύο metrics κοιτάζουν **διαφορετικό πράγμα**.

Η **accuracy** κοιτάζει μόνο την τελική απόφαση:

$$\text{red ή white με threshold } 0{,}5.$$

Δηλαδή, αφού το μοντέλο βγάλει μια πιθανότητα $P(\text{white})$, η accuracy εξετάζει αν η τελική ετικέτα που προκύπτει είναι σωστή. Αν $P(\text{white})$ μείνει πάνω από 0,5, το μοντέλο θα προβλέψει white, ακόμη κι αν αυτή η πιθανότητα έχει μειωθεί αρκετά.

Η **AUC**, αντίθετα, δεν ενδιαφέρεται για το αν τελικά περάσαμε ή όχι το συγκεκριμένο threshold 0,5. Ενδιαφέρεται για τη **σειρά** των πιθανοτήτων. Ρωτάει κάτι σαν:

> Αν πάρω τυχαία ένα white και ένα red κρασί, δίνει το μοντέλο μεγαλύτερη πιθανότητα white στο πραγματικό white από ό,τι στο πραγματικό red;

Όταν η AUC είναι 0,9220, αυτό σημαίνει ότι αυτή η σειρά είναι σωστή σε πολύ μεγάλο βαθμό. Με απλά λόγια, τα `chlorides` βοηθούν το μοντέλο να καταλάβει ποια κρασιά μοιάζουν περισσότερο με white και ποια μοιάζουν περισσότερο με red.

Το πρόβλημα είναι ότι το default threshold 0,5 είναι αρκετά αυστηρό για να προβλεφθεί red. Από την εξίσωση του μοντέλου:

$$P(\text{white} \mid chlorides) = \frac{1}{1 + e^{-(1{,}978076 - 14{,}922372 \cdot chlorides)}}$$

το μοντέλο θα περάσει από πρόβλεψη white σε πρόβλεψη red όταν:

$$1{,}978076 - 14{,}922372 \cdot chlorides = 0.$$

Άρα:

$$chlorides \approx \frac{1{,}978076}{14{,}922372} \approx 0{,}133.$$

Δηλαδή, για να προβλέψει το μοντέλο `red`, πρέπει τα `chlorides` να είναι περίπου πάνω από 0,133. Όμως πολλά κόκκινα κρασιά έχουν μεν υψηλότερα chlorides από τα λευκά, αλλά όχι τόσο υψηλά ώστε η πιθανότητα $P(\text{white})$ να πέσει κάτω από 0,5. Έτσι, το μοντέλο τα βάζει σε σωστότερη σειρά, άρα η AUC είναι υψηλή, αλλά στο τελικό threshold συνεχίζει να τα χαρακτηρίζει ως white, άρα η accuracy και ειδικά το recall red μένουν χαμηλά.

Συνεπώς, τα `chlorides` δεν είναι "κακή" μεταβλητή. Αντίθετα, είναι πολύ καλή μεταβλητή για να διαχωρίσει τις δύο κλάσεις ως προς τη σειρά των πιθανοτήτων. Αυτό που δεν λειτουργεί ιδανικά είναι το συγκεκριμένο κατώφλι απόφασης 0,5 μέσα σε ένα ανισόρροπο dataset με πολύ περισσότερα white από red.

4.4.4 Ερμηνεία με απλά λόγια

Τα chlorides είναι η πιο χρήσιμη από τις τρεις μεταβλητές ως προς τη διακριτική ικανότητα. Όσο αυξάνονται, τόσο το κρασί μοιάζει περισσότερο με red. Ενδεικτικά:

chlorides	0,031	0,038	0,047	0,065	0,087
$P(\text{white})$	0,8199	0,8039	0,7819	0,7327	0,6637

Η πιθανότητα για white πέφτει καθώς αυξάνονται τα chlorides. Όμως ακόμη και στο 90ό εκατοστημόριο, δηλαδή περίπου στο 0,087, η πιθανότητα παραμένει πάνω από 0,5. Άρα με το default threshold το μοντέλο συνεχίζει να προβλέπει white για πολλά κρασιά που ίσως έχουν ήδη "κόκκινο" προφίλ.

Αν η εργασία μάς ζητούσε βελτιστοποίηση κατωφλιού, θα μπορούσαμε να μελετήσουμε διαφορετικό threshold. Εδώ όμως, για το υποερώτημα A(a), κρατήσαμε την τυπική πρόβλεψη της Logistic Regression.

5 Συμπέρασμα για το υποερώτημα A(a)

Με βάση τη Logistic Regression και μία μεταβλητή κάθε φορά:

1. Το `alcohol` είναι ουσιαστικά ακατάλληλο ως μοναδική μεταβλητή. Το μοντέλο προβλέπει όλα τα test δείγματα ως white. Η balanced accuracy είναι 0,5 και η AUC είναι 0,5219, δηλαδή σχεδόν τυχαία.

2. Το `pH` έχει κάποια πληροφορία. Η AUC ανεβαίνει στο 0,6798 και το μοντέλο αρχίζει να εντοπίζει λίγα red δείγματα. Όμως το recall red παραμένει πολύ χαμηλό, μόλις 0,0921.

3. Τα `chlorides` είναι η πιο ισχυρή μεταβλητή ως προς τη διακριτική ικανότητα. Η AUC φτάνει το 0,9220, πολύ υψηλότερα από τις άλλες δύο μεταβλητές. Παρ' όλα αυτά, με το default κατώφλι 0,5 η Logistic Regression εξακολουθεί να προβλέπει κυρίως white, οπότε το recall red μένει χαμηλό.

Άρα, αν μας ενδιαφέρει ποια μεταβλητή ξεχωρίζει καλύτερα τα δύο είδη κρασιού μέσα στη Logistic Regression, η απάντηση είναι:

$$\boxed{\text{chlorides}}$$

Αν όμως κοιτάμε μόνο τις τελικές προβλέψεις με threshold 0,5, πρέπει να είμαστε προσεκτικοί: η απλή accuracy δεν λέει όλη την ιστορία, επειδή τα δεδομένα έχουν πολύ περισσότερα white από red κρασιά.

6 Υποερώτημα A(b): Linear Discriminant Analysis

6.1 Τι ζητάει το υποερώτημα A(b)

Το υποερώτημα A(b) ζητά να κάνουμε ακριβώς την ίδια διαδικασία με πριν, αλλά αντί για Logistic Regression να χρησιμοποιήσουμε τη μέθοδο **Linear Discriminant Analysis**, συντομογραφικά **LDA**.

Δηλαδή, ξαναπαίρνουμε τις τρεις μεταβλητές μία-μία:

`alcohol`, `pH`, `chlorides`

και εκπαιδεύουμε τρία ξεχωριστά μοντέλα LDA:

1. LDA με μοναδική μεταβλητή το alcohol
2. LDA με μοναδική μεταβλητή το pH
3. LDA με μοναδική μεταβλητή τα chlorides

Ο στόχος παραμένει ίδιος:

$$\text{να προβλέψουμε αν ένα κρασί είναι } red \text{ ή } white.$$

Χρησιμοποιούμε και πάλι τον ίδιο διαχωρισμό:

$$70\% \text{ training}, \qquad 30\% \text{ test},$$

με:

`random_state = 6931`

και με την ίδια κωδικοποίηση:

$$red = 0, \qquad white = 1.$$

6.2 Θεωρητικό υπόβαθρο: τι κάνει η LDA

Η LDA είναι μια μέθοδος ταξινόμησης. Όπως και η Logistic Regression, στο τέλος θέλει να απαντήσει:

red ή white;

Όμως σκέφτεται το πρόβλημα με διαφορετικό τρόπο.

Η Logistic Regression προσπαθεί να μάθει απευθείας μια σχέση ανάμεσα στη μεταβλητή x και στην πιθανότητα $P(\text{white} \mid x)$. Η LDA, από την άλλη, προσπαθεί πρώτα να περιγράψει **πώς κατανέμονται οι τιμές της μεταβλητής μέσα σε κάθε κλάση**.

Με πιο απλά λόγια, για κάθε μεταβλητή η LDA ρωτά:

Πού βρίσκονται συνήθως οι τιμές των red κρασιών και πού βρίσκονται συνήθως οι τιμές των white κρασιών;

Για παράδειγμα, για τα `chlorides`, η LDA κοιτάζει:

- ποια είναι η μέση τιμή των `chlorides` στα red κρασιά,
- ποια είναι η μέση τιμή των `chlorides` στα white κρασιά,
- πόσο απλωμένες είναι οι τιμές γύρω από αυτούς τους μέσους όρους,
- πόσο συχνή είναι κάθε κλάση στο training set.

6.2.1 Η βασική υπόθεση της LDA

Η LDA υποθέτει ότι, μέσα σε κάθε κλάση, οι τιμές της προβλεπτικής μεταβλητής μοιάζουν περίπου με κανονική κατανομή. Για μία μόνο μεταβλητή, αυτό σημαίνει ότι φανταζόμαστε δύο καμπύλες:

μία καμπύλη για τα red, μία καμπύλη για τα white.

Η LDA επίσης υποθέτει ότι οι δύο κλάσεις έχουν περίπου **ίδια διασπορά** για τη συγκεκριμένη μεταβλητή. Αυτό είναι σημαντικό: η LDA επιτρέπει οι δύο κλάσεις να έχουν διαφορετικό κέντρο, δηλαδή διαφορετική μέση τιμή, αλλά χρησιμοποιεί κοινή εκτίμηση της διασποράς.

Γι' αυτό ονομάζεται **Linear** Discriminant Analysis. Με μία μεταβλητή, το αποτέλεσμα είναι ένα απλό όριο πάνω στον άξονα της μεταβλητής:

αν $x <$ όριο $\Rightarrow$ μία κλάση, αν $x >$ όριο $\Rightarrow$ η άλλη κλάση.

6.2.2 Πώς αποφασίζει η LDA

Η LDA υπολογίζει για κάθε κλάση ένα σκορ. Για μία μεταβλητή x, το σκορ της κλάσης k έχει την ιδέα:

$$\delta_k(x) = \frac{x\mu_k}{s^2} - \frac{\mu_k^2}{2s^2} + \log(\pi_k).$$

Οι ποσότητες αυτές σημαίνουν:

- μ_k: η μέση τιμή της μεταβλητής στην κλάση k,
- s^2: η κοινή διασπορά που χρησιμοποιεί η LDA,
- π_k: το prior της κλάσης, δηλαδή πόσο συχνή είναι η κλάση στο training set.

Δεν χρειάζεται να απομνημονεύσουμε τον τύπο. Αυτό που χρειάζεται να καταλάβουμε είναι το εξής:

Η LDA συγκρίνει πόσο καλά ταιριάζει η τιμή x στο προφίλ των red και πόσο καλά ταιριάζει στο προφίλ των white.

Στο τέλος επιλέγει την κλάση με το μεγαλύτερο σκορ.

Για να το κάνουμε πιο συγκεκριμένο, ας σκεφτούμε ότι η LDA δίνει σε κάθε κρασί δύο βαθμολογίες:

$$\text{score}_{red}(x), \qquad \text{score}_{white}(x).$$

Η πρώτη βαθμολογία λέει πόσο ταιριάζει η τιμή x με το προφίλ των red κρασιών. Η δεύτερη λέει πόσο ταιριάζει η ίδια τιμή x με το προφίλ των white κρασιών. Αν το

score του white είναι μεγαλύτερο, το μοντέλο προβλέπει `white`. Αν το score του red είναι μεγαλύτερο, προβλέπει `red`.

Επειδή εδώ έχουμε μόνο δύο κλάσεις, δεν χρειάζεται να κοιτάμε ξεχωριστά τις δύο βαθμολογίες. Μπορούμε να κοιτάμε τη διαφορά τους:

$$\text{score}(x) = \text{score}_{white}(x) - \text{score}_{red}(x).$$

Αυτή η διαφορά λειτουργεί σαν μια **ζυγαριά**. Αν είναι θετική, γέρνει προς `white`. Αν είναι αρνητική, γέρνει προς `red`. Στην LDA, όταν έχουμε μία μόνο προβλεπτική μεταβλητή, αυτή η διαφορά μπορεί να γραφτεί ως μια απλή γραμμική σχέση:

$$\text{score}(x) = \beta_0 + \beta_1 x.$$

Το β_0 είναι μια σταθερά, ενώ το β_1 δείχνει πόσο αλλάζει το score όταν αλλάζει η τιμή της μεταβλητής x. Το σημαντικό είναι ότι το score δεν είναι πιθανότητα. Είναι απλώς ένας αριθμός που μας λέει προς ποια κλάση γέρνει η απόφαση.

Άρα ο κανόνας απόφασης είναι:

$$\begin{cases} \text{white}, & \text{αν } \beta_0 + \beta_1 x \geq 0, \\ \text{red}, & \text{αν } \beta_0 + \beta_1 x < 0. \end{cases}$$

Το σημείο όπου το score γίνεται ακριβώς μηδέν είναι το σημείο όπου το μοντέλο είναι οριακά ανάμεσα στις δύο αποφάσεις. Αυτό το σημείο λέγεται **όριο απόφασης**:

$$x^* = -\frac{\beta_0}{\beta_1}.$$

Με μία μεταβλητή, το όριο απόφασης είναι απλώς ένας αριθμός πάνω στον άξονα της μεταβλητής. Για παράδειγμα, αν το όριο στα `chlorides` είναι 0,094, τότε τιμές κάτω από αυτό το σημείο οδηγούν σε πρόβλεψη `white`, ενώ τιμές πάνω από αυτό οδηγούν σε πρόβλεψη `red`.

6.3 Πώς λύσαμε το υποερώτημα A(b)

Η διαδικασία στον κώδικα είναι σχεδόν ίδια με τη Logistic Regression. Αλλάζουμε μόνο το μοντέλο που εκπαιδεύουμε:

```
from sklearn.discriminant_analysis import LinearDiscriminantAnalysis

model = LinearDiscriminantAnalysis()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
```

Το `y_pred` περιέχει τις **τελικές προβλέψεις** του μοντέλου. Δηλαδή για κάθε κρασί κρατάει μόνο την τελική απόφαση:

`red` ή `white`.

Το `y_proba`, αντίθετα, δεν κρατά μόνο την τελική απόφαση. Κρατά την **πιθανότητα** που δίνει το μοντέλο για κάθε κλάση. Για παράδειγμα, για ένα κρασί το μοντέλο μπορεί να δώσει:

$$P(\text{red}) = 0{,}28, \qquad P(\text{white}) = 0{,}72.$$

Σε αυτή την περίπτωση, η τελική πρόβλεψη θα είναι `white`, επειδή η πιθανότητα για `white` είναι μεγαλύτερη.

Η εντολή:

```
y_proba = model.predict_proba(X_test)[:, 1]
```

σημαίνει ότι από τον πίνακα πιθανοτήτων κρατάμε μόνο τη δεύτερη στήλη, δηλαδή την πιθανότητα για την κλάση `white`. Αυτό γίνεται επειδή έχουμε ορίσει:

$$red = 0, \qquad white = 1.$$

Άρα η στήλη `[:, 1]` αντιστοιχεί στο:

$$P(\text{white}).$$

Χρειαζόμαστε αυτές τις πιθανότητες για την **AUC**, επειδή η AUC δεν κοιτάζει μόνο αν η τελική πρόβλεψη ήταν `red` ή `white`. Κοιτάζει αν το μοντέλο βάζει τα κρασιά σε σωστή σειρά: τα πραγματικά `white` πρέπει γενικά να έχουν υψηλότερη πιθανότητα $P(\text{white})$, ενώ τα πραγματικά `red` πρέπει γενικά να έχουν χαμηλότερη πιθανότητα $P(\text{white})$. Γι' αυτό η AUC υπολογίζεται από το `y_proba` και όχι μόνο από το `y_pred`.

6.4 Κέντρα και διασπορά των δύο κλάσεων στο training set

Πριν δούμε τα τελικά αποτελέσματα, αξίζει να δούμε όχι μόνο τις μέσες τιμές που βλέπει η LDA στο training set, αλλά και τη **διασπορά** των τιμών. Η μέση τιμή μάς λέει πού βρίσκεται το κέντρο κάθε κλάσης. Η τυπική απόκλιση μάς λέει πόσο απλωμένες είναι οι τιμές γύρω από αυτό το κέντρο.

Μεταβλητή	Mean red	SD red	Mean white	SD white
alcohol	10,4285	1,0132	10,5535	1,1413
pH	3,2953	0,1487	3,1970	0,1431
chlorides	0,0855	0,0495	0,0462	0,0226

Αυτός ο πίνακας είναι πιο χρήσιμος από έναν πίνακα μόνο με μέσες τιμές, γιατί μια διαφορά μέσων έχει νόημα μόνο σε σχέση με το πόσο απλωμένες είναι οι τιμές. Αν δύο κλάσεις έχουν λίγο διαφορετικά κέντρα αλλά πολύ μεγάλη διασπορά, τότε οι τιμές τους θα επικαλύπτονται πολύ και ο διαχωρισμός θα είναι δύσκολος.

Με βάση τον πίνακα:

- Στο `alcohol`, οι μέσες τιμές red και white είναι πολύ κοντά. Η διαφορά τους είναι περίπου $0{,}125$, ενώ οι τυπικές αποκλίσεις είναι περίπου $1{,}0$ και $1{,}14$. Δηλαδή το άπλωμα των τιμών είναι πολύ μεγαλύτερο από τη διαφορά των κέντρων. Άρα οι δύο κλάσεις επικαλύπτονται έντονα και δεν περιμένουμε καλό διαχωρισμό.

- Στο `pH`, τα red έχουν λίγο υψηλότερη μέση τιμή από τα white. Η διαφορά των μέσων είναι περίπου $0{,}098$, ενώ οι τυπικές αποκλίσεις είναι περίπου $0{,}14$–$0{,}15$. Εδώ η διαφορά είναι πιο ουσιαστική σε σχέση με τη διασπορά, άρα υπάρχει κάποια πληροφορία, αλλά οι δύο κατανομές εξακολουθούν να επικαλύπτονται αρκετά.

- Στα `chlorides`, η διαφορά είναι πολύ μεγαλύτερη: τα red έχουν περίπου διπλάσια μέση τιμή από τα white. Η διαφορά των μέσων είναι περίπου $0{,}039$. Παρότι η διασπορά των red είναι αρκετά μεγάλη $(0{,}0495)$, τα white είναι πιο συγκεντρωμένα γύρω από χαμηλές τιμές $(SD = 0{,}0226)$. Άρα τα `chlorides` δίνουν πολύ καλύτερη διακριτική πληροφορία από τις άλλες δύο μεταβλητές.

6.5 Συνοπτικά αποτελέσματα LDA

Μεταβλητή	Accuracy	Bal. Acc.	AUC	Recall white	Recall red	F1 red
alcohol	0,7539	0,5000	0,5219	1,0000	0,0000	0,0000
pH	0,7527	0,5336	0,6798	0,9649	0,1023	0,1691
chlorides	0,7804	0,5864	0,9220	0,9683	0,2046	0,3143

Για να διαβάσουμε τον πίνακα, θυμόμαστε ξανά:

- **Accuracy**: συνολικό ποσοστό σωστών προβλέψεων.
- **Balanced Accuracy**: πιο δίκαιο μέτρο όταν οι κλάσεις είναι ανισόρροπες.
- **AUC**: πόσο καλά μπαίνουν τα κρασιά σε σωστή σειρά πιθανοτήτων, ανεξάρτητα από το τελικό threshold.
- **Recall white**: πόσα από τα πραγματικά white βρήκαμε.
- **Recall red**: πόσα από τα πραγματικά red βρήκαμε.
- **F1 red**: συνδυασμός precision και recall για την κλάση red.

Το βασικό μήνυμα του πίνακα είναι ότι η LDA με `chlorides` είναι η καλύτερη από τις τρεις LDA δοκιμές. Έχει τη μεγαλύτερη accuracy, τη μεγαλύτερη balanced accuracy, το μεγαλύτερο recall red και το μεγαλύτερο F1 red. Παράλληλα, η AUC παραμένει πολύ υψηλή, 0,9220.

6.6 Όρια απόφασης της LDA

Για να καταλάβουμε καλύτερα τις τελικές προβλέψεις, κοιτάμε και το όριο απόφασης κάθε μοντέλου:

Μεταβλητή	β_0	β_1	Όριο απόφασης x^*
alcohol	0,055731	0,101268	-0,550
pH	16,392779	-4,705414	3,484
chlorides	3,736194	-39,764907	0,094

Το όριο x^* είναι η τιμή της μεταβλητής όπου η LDA αλλάζει από τη μία πρόβλεψη στην άλλη.

Για να διαβάσουμε σωστά αυτόν τον πίνακα, θυμόμαστε ότι η LDA γράφει την απόφαση ως:

$$\text{score}(x) = \beta_0 + \beta_1 x.$$

Το β_0 είναι η σταθερά του μοντέλου. Το β_1 δείχνει προς ποια κατεύθυνση κινείται το score όταν αυξάνεται η τιμή της μεταβλητής. Αν το score είναι θετικό, η LDA προβλέπει `white`. Αν είναι αρνητικό, προβλέπει `red`. Το όριο απόφασης είναι το σημείο όπου:

$$\beta_0 + \beta_1 x = 0.$$

Αν λύσουμε αυτή την εξίσωση ως προς x, παίρνουμε:

$$x^* = -\frac{\beta_0}{\beta_1}.$$

Άρα το x^* είναι το σημείο ισορροπίας. Πριν από αυτό το σημείο το μοντέλο γέρνει προς τη μία κλάση, μετά από αυτό το σημείο γέρνει προς την άλλη.

Ο πίνακας διαβάζεται ως εξής:

- Για το `alcohol`, το όριο είναι $-0{,}550$. Αυτό δεν έχει πρακτικό νόημα ως πραγματική τιμή alcohol, γιατί τα ποσοστά αλκοόλ είναι θετικά και πολύ μεγαλύτερα από $-0{,}550$. Επειδή το β_1 είναι θετικό, για όλες τις πραγματικές τιμές alcohol το score μένει θετικό. Άρα η LDA προβλέπει σχεδόν πάντα `white`. Γι' αυτό το recall red είναι 0.
- Για το `pH`, το όριο είναι $3{,}484$. Εδώ το β_1 είναι αρνητικό. Αυτό σημαίνει ότι όσο αυξάνεται το pH, το score μειώνεται και το μοντέλο κινείται προς την πρόβλεψη `red`. Άρα η LDA προβλέπει κυρίως `white` για τιμές pH κάτω από 3,484 και `red` για τιμές πάνω από 3,484. Επειδή όμως πολλές τιμές pH και των δύο κλάσεων βρίσκονται κοντά μεταξύ τους, ο διαχωρισμός παραμένει μέτριος.
- Για τα `chlorides`, το όριο είναι $0{,}094$. Και εδώ το β_1 είναι αρνητικό, άρα όσο αυξάνονται τα chlorides, το μοντέλο κινείται προς `red`. Το όριο 0,094 σημαίνει ότι χαμηλότερες τιμές chlorides οδηγούν κυρίως σε πρόβλεψη `white`, ενώ υψηλότερες τιμές οδηγούν σε πρόβλεψη `red`. Αυτό ταιριάζει με τα δεδομένα, γιατί τα red κρασιά έχουν υψηλότερα chlorides από τα white.

Επομένως, ο πίνακας των ορίων απόφασης δεν είναι απλώς ένας τεχνικός πίνακας με συντελεστές. Μας λέει **πού ακριβώς βάζει η LDA τη γραμμή** που χωρίζει το `white` από το `red` για κάθε μεταβλητή.

6.7 Πείραμα 1: LDA με μεταβλητή alcohol

6.7.1 Το μοντέλο που εκπαιδεύτηκε

Για το `alcohol`, το γραμμικό σκορ της LDA είναι:

$$\text{score}(alcohol) = 0{,}055731 + 0{,}101268 \cdot alcohol.$$

Το όριο απόφασης είναι:

$$alcohol^* \approx -0{,}550.$$

Αυτό το όριο είναι έξω από το πραγματικό εύρος της μεταβλητής, γιατί τα ποσοστά αλκοόλ των κρασιών είναι θετικά και βρίσκονται περίπου γύρω από 8 έως 14. Άρα για όλες τις πραγματικές τιμές alcohol, το σκορ είναι θετικό και το μοντέλο προβλέπει `white`.

6.7.2 Confusion matrix

	Predicted red	Predicted white
True red	0	391
True white	0	1198

Η LDA με `alcohol` δεν εντοπίζει κανένα κόκκινο κρασί. Όπως και η Logistic Regression με `alcohol`, λειτουργεί πρακτικά σαν ταξινομητής που λέει πάντα `white`.

6.7.3 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7539
Balanced Accuracy	0,5000
AUC	0,5219
Recall white	1,0000
Recall red	0,0000
F1 red	0,0000

Η accuracy 0,7539 δεν είναι ένδειξη καλής ταξινόμησης. Είναι απλώς το ποσοστό των white στο test set. Το recall red είναι 0, άρα το μοντέλο δεν έχει πρακτική χρησιμότητα για να ξεχωρίσει red από white με βάση μόνο το `alcohol`.

6.8 Πείραμα 2: LDA με μεταβλητή pH

6.8.1 Το μοντέλο που εκπαιδεύτηκε

Για το `pH`, το σκορ είναι:

$$\text{score}(pH) = 16{,}392779 - 4{,}705414 \cdot pH.$$

Το όριο απόφασης είναι:

$$pH^* \approx 3{,}484.$$

Επειδή ο συντελεστής είναι αρνητικός, όσο αυξάνεται το pH, το σκορ μειώνεται. Άρα:

$$\begin{cases} \text{white}, & \text{αν } pH < 3{,}484, \\ \text{red}, & \text{αν } pH > 3{,}484. \end{cases}$$

Αυτό ταιριάζει με τη μέση εικόνα των δεδομένων: τα red κρασιά έχουν υψηλότερο μέσο pH από τα white.

6.8.2 Confusion matrix

	Predicted red	Predicted white
True red	40	351
True white	42	1156

Η LDA με `pH` βρίσκει σωστά 40 από τα 391 red κρασιά. Είναι λίγο καλύτερη από την Logistic Regression με `pH`, η οποία έβρισκε 36 red, αλλά η βελτίωση παραμένει μικρή.

6.8.3 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7527
Balanced Accuracy	0,5336
AUC	0,6798
Recall white	0,9649
Recall red	0,1023
F1 red	0,1691

Η AUC 0,6798 δείχνει ότι το `pH` περιέχει κάποια πληροφορία, αλλά όχι αρκετή για καθαρό διαχωρισμό. Το recall red παραμένει χαμηλό, περίπου 10,23%. Άρα το pH βοηθά, αλλά δεν είναι αρκετό από μόνο του.

6.9 Πείραμα 3: LDA με μεταβλητή chlorides

6.9.1 Το μοντέλο που εκπαιδεύτηκε

Για τα `chlorides`, το σκορ είναι:

$$\text{score}(chlorides) = 3{,}736194 - 39{,}764907 \cdot chlorides.$$

Το όριο απόφασης είναι:

$$chlorides^* \approx 0{,}094.$$

Επειδή ο συντελεστής είναι αρνητικός, όσο αυξάνονται τα `chlorides`, το σκορ μειώνεται και το μοντέλο κινείται προς την πρόβλεψη `red`. Άρα:

$$\begin{cases} \text{white}, & \text{αν } chlorides < 0{,}094, \\ \text{red}, & \text{αν } chlorides > 0{,}094. \end{cases}$$

Αυτό έχει καθαρή ερμηνεία: τα κόκκινα κρασιά έχουν υψηλότερα chlorides από τα λευκά, οπότε όταν η τιμή ξεπεράσει περίπου το 0,094, η LDA αρχίζει να θεωρεί το κρασί πιο πιθανό red.

6.9.2 Confusion matrix

	Predicted red	Predicted white
True red	80	311
True white	38	1160

Αυτός είναι ο καλύτερος confusion matrix από τις τρεις LDA δοκιμές:

- Βρίσκουμε σωστά 80 red κρασιά, ενώ με `pH` βρίσκαμε 40 και με `alcohol` 0.
- Χάνουμε ακόμα 311 red κρασιά, άρα το μοντέλο δεν είναι τέλειο.
- Κάνουμε μόνο 38 λάθη στα white, δηλαδή τα περισσότερα white εξακολουθούν να ταξινομούνται σωστά.

6.9.3 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7804
Balanced Accuracy	0,5864
AUC	0,9220
Recall white	0,9683
Recall red	0,2046
F1 red	0,3143

Τα `chlorides` δίνουν την καλύτερη επίδοση στη LDA. Η accuracy ανεβαίνει στο 0,7804 και το recall red ανεβαίνει στο 0,2046. Αυτό σημαίνει ότι η LDA με `chlorides` βρίσκει περίπου 20,46% των κόκκινων κρασιών. Το ποσοστό δεν είναι τεράστιο, αλλά είναι σαφώς καλύτερο από το 0% του `alcohol` και το 10,23% του `pH`.

6.9.4 Γιατί βελτιώνεται η LDA με chlorides;

Στη Logistic Regression με `chlorides`, είδαμε ότι το όριο για να προβλεφθεί `red` ήταν περίπου:

$$chlorides \approx 0{,}133.$$

Στην LDA, το αντίστοιχο όριο είναι:

$$chlorides \approx 0{,}094.$$

Άρα η LDA είναι λιγότερο αυστηρή στο να προβλέψει `red`. Δεν απαιτεί τόσο υψηλή τιμή chlorides όσο η Logistic Regression. Αυτό έχει άμεση συνέπεια:

- περισσότερα πραγματικά red περνούν στην περιοχή πρόβλεψης red,
- το recall red ανεβαίνει από 0,0665 σε 0,2046,
- το F1 red ανεβαίνει από 0,1182 σε 0,3143,
- η accuracy ανεβαίνει από 0,7558 σε 0,7804.

Παρατηρούμε όμως ότι η AUC παραμένει σχεδόν ίδια. Αυτό συμβαίνει επειδή και οι δύο μέθοδοι, με μία μόνο μεταβλητή, βάζουν τα δείγματα σχεδόν στην ίδια σειρά ως προς τα `chlorides`. Η διαφορά τους δεν είναι τόσο στη σειρά των πιθανοτήτων, αλλά στο σημείο όπου βάζουν το τελικό όριο απόφασης.

6.10 Συμπέρασμα για το υποερώτημα A(b)

Με βάση τη Linear Discriminant Analysis:

1. Το `alcohol` δεν βοηθά ουσιαστικά. Η LDA προβλέπει όλα τα δείγματα ως `white`, άρα έχει balanced accuracy 0,5 και recall red 0.
2. Το `pH` δίνει μια μικρή βελτίωση. Αναγνωρίζει λίγα red κρασιά, αλλά το recall red παραμένει χαμηλό.
3. Τα `chlorides` είναι ξεκάθαρα η καλύτερη μεταβλητή για την LDA. Έχουν AUC 0,9220 και δίνουν την καλύτερη accuracy, balanced accuracy, recall red και F1 red.

Άρα, για το υποερώτημα A(b), η καλύτερη μοναδική προβλεπτική μεταβλητή είναι:

$$\boxed{\text{chlorides}}$$

Η LDA με `chlorides` δεν λύνει τέλεια το πρόβλημα, επειδή εξακολουθεί να χάνει αρκετά κόκκινα κρασιά. Όμως σε σχέση με τη Logistic Regression, βάζει πιο χρήσιμο όριο απόφασης για τη συγκεκριμένη μεταβλητή και έτσι βελτιώνει αισθητά την αναγνώριση της κλάσης red.

7 Υποερώτημα A(c): Gaussian Naive Bayes

7.1 Τι ζητάει το υποερώτημα A(c)

Το υποερώτημα A(c) ζητά να κάνουμε ξανά πρόβλεψη του τύπου κρασιού χρησιμοποιώντας μία μεταβλητή κάθε φορά:

`alcohol`, `pH`, `chlorides`.

Αυτή τη φορά η μέθοδος είναι το **Naive Bayes**, με την υπόθεση ότι τα χαρακτηριστικά σε κάθε κλάση ακολουθούν **κανονική κατανομή**. Επειδή χρησιμοποιούμε κανονική κατανομή, η μέθοδος λέγεται συχνά **Gaussian Naive Bayes** ή συντομογραφικά **GNB**.

Όπως και πριν, φτιάχνουμε τρία μοντέλα:

1. Gaussian NB με μοναδική μεταβλητή το alcohol
2. Gaussian NB με μοναδική μεταβλητή το pH
3. Gaussian NB με μοναδική μεταβλητή τα chlorides

7.2 Θεωρητικό υπόβαθρο: η ιδέα του Bayes

Ο Naive Bayes βασίζεται στον κανόνα του Bayes. Η ιδέα του Bayes είναι ένας τρόπος να **ενημερώνουμε την άποψή μας** όταν βλέπουμε μια νέα πληροφορία.

Στο δικό μας πρόβλημα, πριν κοιτάξουμε οποιαδήποτε χημική μεταβλητή, ξέρουμε ήδη κάτι απλό: στο dataset υπάρχουν πολύ περισσότερα white κρασιά από red. Άρα, αν δεν ξέραμε τίποτα άλλο για ένα κρασί, η πιο λογική αρχική μας υπόθεση θα ήταν ότι μάλλον είναι `white`.

Αυτή είναι η **αρχική πληροφορία** μας.

Μετά όμως βλέπουμε μια τιμή, για παράδειγμα τα `chlorides`. Αν τα `chlorides` είναι πολύ υψηλά, τότε αυτό είναι ένδειξη υπέρ του `red`, επειδή τα κόκκινα κρασιά τείνουν να έχουν υψηλότερα chlorides. Άρα αλλάζουμε την αρχική μας άποψη.

Με απλά λόγια, ο Bayes κάνει το εξής:

> Ξεκινάω με το πόσο συχνή είναι κάθε κλάση. Μετά βλέπω την τιμή της μεταβλητής και αναπροσαρμόζω την πιθανότητα για κάθε κλάση.

Η βασική ερώτηση είναι:

> Αν ξέρω την τιμή μιας μεταβλητής x, ποια είναι η πιθανότητα το κρασί να είναι `red` και ποια η πιθανότητα να είναι `white`;

Με μαθηματικό συμβολισμό θέλουμε να υπολογίσουμε:

$$P(\text{red} \mid x) \qquad \text{και} \qquad P(\text{white} \mid x).$$

Ο τύπος του Bayes λέει:

$$P(\text{κλάση} \mid x) = \frac{P(x \mid \text{κλάση})P(\text{κλάση})}{P(x)}.$$

Αυτός ο τύπος μπορεί να φαίνεται βαρύς, αλλά τα κομμάτια του έχουν πολύ απλή ερμηνεία:

- $P(\text{κλάση})$: πόσο συχνή είναι η κλάση πριν δούμε τη μεταβλητή. Αυτό λέγεται **prior**. Είναι η αρχική μας γνώμη. Στα δεδομένα μας τα white είναι περισσότερα, άρα πριν δούμε το x, το μοντέλο έχει αρχική τάση προς `white`.
- $P(x \mid \text{κλάση})$: πόσο πιθανό είναι να δούμε την τιμή x αν ξέρουμε ότι το κρασί ανήκει σε αυτή την κλάση. Αυτό λέγεται **likelihood**. Είναι το πόσο ταιριάζει η συγκεκριμένη τιμή με το προφίλ κάθε κλάσης. Για παράδειγμα, μια υψηλή τιμή `chlorides` ταιριάζει περισσότερο στο προφίλ των red.
- $P(\text{κλάση} \mid x)$: η πιθανότητα της κλάσης αφού έχουμε δει την τιμή x. Αυτό λέγεται **posterior**. Είναι η τελική ενημερωμένη πιθανότητα που χρησιμοποιεί το μοντέλο για την πρόβλεψη.
- $P(x)$: η συνολική πιθανότητα να εμφανιστεί η τιμή x, ανεξάρτητα από το αν το κρασί είναι red ή white. Για την τελική απόφαση δεν μας ενδιαφέρει πολύ, γιατί είναι ίδια και στη σύγκριση με red και στη σύγκριση με white.

Ένας απλός τρόπος να το θυμόμαστε είναι:

$$\text{τελική πιθανότητα} \propto \text{πόσο συχνή είναι η κλάση} \times \text{πόσο ταιριάζει η τιμή με την κλάση}.$$

Δηλαδή:

$$\text{posterior} \propto \text{prior} \times \text{likelihood}.$$

Στην ταξινόμηση δεν χρειάζεται να υπολογίσουμε τέλεια όλες τις ποσότητες του τύπου. Επειδή το $P(x)$ είναι ίδιο και για τις δύο κλάσεις, αρκεί να συγκρίνουμε:

$$P(x \mid red)P(red) \quad \text{με} \quad P(x \mid white)P(white).$$

Αν το πρώτο είναι μεγαλύτερο, προβλέπουμε `red`. Αν το δεύτερο είναι μεγαλύτερο, προβλέπουμε `white`.

Παράδειγμα με απλά λόγια:

- Αν ένα κρασί έχει χαμηλά `chlorides`, αυτή η τιμή ταιριάζει περισσότερο με τα white. Επειδή τα white είναι και πιο συχνά στο dataset, το μοντέλο συνήθως θα προβλέψει `white`.
- Αν ένα κρασί έχει υψηλά `chlorides`, αυτή η τιμή ταιριάζει περισσότερο με τα red. Τότε το likelihood υπέρ του red μπορεί να γίνει αρκετά μεγάλο ώστε να ξεπεράσει το αρχικό πλεονέκτημα των white και το μοντέλο να προβλέψει `red`.

7.3 Γιατί λέγεται Gaussian Naive Bayes

Η λέξη **Gaussian** σημαίνει ότι υποθέτουμε κανονική κατανομή. Δηλαδή, για κάθε κλάση, φανταζόμαστε ότι οι τιμές της μεταβλητής σχηματίζουν μια καμπύλη τύπου καμπάνας.

Για παράδειγμα, για τα `chlorides`, το μοντέλο υποθέτει:

$$chlorides \mid red \sim N(\mu_{red}, \sigma^2_{red})$$

και

$$chlorides \mid white \sim N(\mu_{white}, \sigma^2_{white}).$$

Σε απλά λόγια:

- τα red έχουν δικό τους μέσο όρο και δική τους διασπορά,
- τα white έχουν δικό τους μέσο όρο και δική τους διασπορά.

Αυτό είναι σημαντική διαφορά από την LDA. Η LDA χρησιμοποιεί κοινή διασπορά για τις δύο κλάσεις. Το Gaussian NB επιτρέπει διαφορετική διασπορά ανά κλάση. Αυτό είναι ιδιαίτερα χρήσιμο όταν η μία κλάση είναι πιο απλωμένη από την άλλη.

Η λέξη **Naive** εμφανίζεται επειδή, όταν έχουμε πολλές μεταβλητές, η μέθοδος υποθέτει ότι οι μεταβλητές είναι ανεξάρτητες μεταξύ τους μέσα σε κάθε κλάση. Στο δικό μας υποερώτημα όμως χρησιμοποιούμε **μία μόνο** μεταβλητή κάθε φορά. Άρα η υπόθεση ανεξαρτησίας δεν δημιουργεί πρακτικό πρόβλημα εδώ.

7.4 Πώς αποφασίζει το Gaussian NB

Για κάθε τιμή x, το μοντέλο υπολογίζει δύο posterior πιθανότητες:

$$P(red \mid x) \qquad \text{και} \qquad P(white \mid x).$$

Έπειτα διαλέγει τη μεγαλύτερη:

$$\begin{cases} \text{white}, & \text{αν } P(white \mid x) > P(red \mid x), \\ \text{red}, & \text{αν } P(red \mid x) > P(white \mid x). \end{cases}$$

Επειδή οι δύο κλάσεις μπορούν να έχουν διαφορετικές διασπορές, το όριο απόφασης δεν είναι πάντα τόσο απλό όσο στην LDA. Στην LDA, με μία μεταβλητή, το όριο προκύπτει από γραμμικό score. Στο Gaussian NB, επειδή έχουμε κανονικές καμπύλες με πιθανώς διαφορετικά πλάτη, η σύγκριση μπορεί να δώσει ένα ή και δύο σημεία αλλαγής. Στα δικά μας πραγματικά εύρη τιμών, όμως, υπάρχει ένα βασικό σχετικό όριο για το `pH` και ένα για τα `chlorides`.

7.5 Πώς λύσαμε το υποερώτημα A(c)

Η υλοποίηση είναι παρόμοια με τις προηγούμενες μεθόδους. Αλλάζουμε μόνο το μοντέλο:

```
from sklearn.naive_bayes import GaussianNB

model = GaussianNB()
model.fit(X_train, y_train)

y_pred = model.predict(X_test)
y_proba = model.predict_proba(X_test)[:, 1]
```

Όπως πριν:

- το `y_pred` περιέχει τις τελικές προβλέψεις `red/white`,
- το `y_proba` περιέχει τις πιθανότητες για `white`,
- το `y_proba` χρησιμοποιείται για την AUC.

7.6 Παράμετροι που έμαθε το Gaussian NB

Το Gaussian NB μαθαίνει για κάθε μεταβλητή και για κάθε κλάση:

- το prior της κλάσης,
- τη μέση τιμή μ,
- την τυπική απόκλιση σ.

Μεταβλητή	μ_{red}	σ_{red}	μ_{white}	σ_{white}
alcohol	10,4285	1,0127	10,5535	1,1411
pH	3,2953	0,1486	3,1970	0,1431
chlorides	0,0855	0,0495	0,0462	0,0226

Τα priors είναι:

$$P(red) = 0{,}2464, \qquad P(white) = 0{,}7536.$$

Ο πίνακας μάς λέει κάτι πολύ σημαντικό:

- Στο `alcohol`, οι μέσες τιμές είναι πολύ κοντά και οι τυπικές αποκλίσεις μεγάλες. Άρα δεν υπάρχει καθαρός διαχωρισμός.
- Στο `pH`, οι μέσες τιμές διαφέρουν περισσότερο σε σχέση με τη διασπορά, άρα υπάρχει κάποια πληροφορία.
- Στα `chlorides`, οι μέσες τιμές διαφέρουν αρκετά και επιπλέον οι διασπορές δεν είναι ίδιες: τα red είναι πολύ πιο απλωμένα από τα white. Αυτό είναι ακριβώς το σημείο όπου το Gaussian NB μπορεί να βοηθήσει, επειδή επιτρέπει διαφορετική διασπορά ανά κλάση.

7.7 Συνοπτικά αποτελέσματα Gaussian NB

Μεταβλητή	Accuracy	Bal. Acc.	AUC	Recall white	Recall red	F1 red
alcohol	0,7539	0,5000	0,5678	1,0000	0,0000	0,0000
pH	0,7520	0,5366	0,6798	0,9608	0,1125	0,1826
chlorides	0,7923	0,6107	0,9214	0,9683	0,2532	0,3750

Ο πίνακας δείχνει καθαρά ότι η καλύτερη μεταβλητή και για το Gaussian NB είναι τα `chlorides`. Έχουν:

- την υψηλότερη accuracy,
- την υψηλότερη balanced accuracy,
- το υψηλότερο recall red,
- το υψηλότερο F1 red,
- και πολύ υψηλή AUC.

7.8 Σχετικά όρια απόφασης

Για να καταλάβουμε τις προβλέψεις, μπορούμε να κοιτάξουμε σε ποιο σημείο περίπου αλλάζει η απόφαση από `white` σε `red`.

Μεταβλητή	Σχετικό όριο απόφασης στο πραγματικό εύρος τιμών
alcohol	δεν υπάρχει χρήσιμο πραγματικό όριο
pH	περίπου 3,475
chlorides	περίπου 0,090

Η ερμηνεία είναι:

- Για το `alcohol`, το μοντέλο δεν βρίσκει πρακτικό όριο που να χωρίζει red και white. Γι' αυτό προβλέπει όλα τα test δείγματα ως `white`.
- Για το `pH`, πάνω από περίπου 3,475 το μοντέλο αρχίζει να προβλέπει `red`. Αυτό είναι κοντά στο αντίστοιχο όριο της LDA.
- Για τα `chlorides`, πάνω από περίπου 0,090 το μοντέλο αρχίζει να προβλέπει `red`. Αυτό είναι λίγο χαμηλότερο από το όριο της LDA (0,094), άρα το Gaussian NB είναι λίγο πιο πρόθυμο να χαρακτηρίσει ένα κρασί ως red.

7.9 Πείραμα 1: Gaussian NB με μεταβλητή alcohol

7.9.1 Confusion matrix

	Predicted red	Predicted white
True red	0	391
True white	0	1198

Όπως και στις άλλες δύο μεθόδους, το `alcohol` δεν βοηθά. Το μοντέλο προβλέπει όλα τα δείγματα ως `white`.

7.9.2 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7539
Balanced Accuracy	0,5000
AUC	0,5678
Recall white	1,0000
Recall red	0,0000
F1 red	0,0000

Η AUC είναι λίγο υψηλότερη από της Logistic Regression και της LDA για το `alcohol`, αλλά η τελική ταξινόμηση παραμένει άχρηστη για την κλάση red, επειδή το recall red είναι 0.

7.10 Πείραμα 2: Gaussian NB με μεταβλητή pH

7.10.1 Confusion matrix

	Predicted red	Predicted white
True red	44	347
True white	47	1151

Το `pH` βρίσκει λίγα περισσότερα red από τη LDA:

$$44 \text{ αντί για } 40.$$

Όμως εξακολουθεί να χάνει τα περισσότερα κόκκινα κρασιά.

7.10.2 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7520
Balanced Accuracy	0,5366
AUC	0,6798
Recall white	0,9608
Recall red	0,1125
F1 red	0,1826

Το `pH` περιέχει κάποια πληροφορία, αλλά όχι αρκετή. Η AUC 0,6798 είναι μέτρια και το recall red παραμένει χαμηλό.

7.11 Πείραμα 3: Gaussian NB με μεταβλητή chlorides

7.11.1 Confusion matrix

	Predicted red	Predicted white
True red	99	292
True white	38	1160

Αυτός είναι ο καλύτερος confusion matrix από όλα τα μοντέλα που έχουμε δει μέχρι τώρα στο Ερώτημα Α:

- Βρίσκει σωστά 99 red κρασιά.
- Χάνει 292 red κρασιά.
- Κάνει 38 λάθη στα white, όπως και η LDA με `chlorides`.
- Κρατά πολύ υψηλή σωστή αναγνώριση των white.

7.11.2 Μέτρα αξιολόγησης

Μέτρο	Τιμή
Accuracy	0,7923
Balanced Accuracy	0,6107
AUC	0,9214
Recall white	0,9683
Recall red	0,2532
F1 red	0,3750

Τα `chlorides` δίνουν καθαρά την καλύτερη επίδοση για το Gaussian NB. Το recall red ανεβαίνει στο 0,2532, δηλαδή το μοντέλο βρίσκει περίπου 25,32% των κόκκινων κρασιών. Δεν είναι τέλειο, αλλά είναι η καλύτερη επίδοση μέχρι τώρα για την κλάση red.

7.11.3 Γιατί το Gaussian NB με chlorides είναι καλύτερο από την LDA;

Η βασική διαφορά είναι ότι η LDA χρησιμοποιεί κοινή διασπορά για red και white, ενώ το Gaussian NB επιτρέπει διαφορετική διασπορά:

$$\sigma_{red} = 0{,}0495, \qquad \sigma_{white} = 0{,}0226.$$

Άρα το Gaussian NB βλέπει ότι:

- τα white είναι πιο συγκεντρωμένα σε χαμηλές τιμές chlorides,
- τα red είναι πιο απλωμένα και μπορούν να εμφανιστούν σε μεγαλύτερο εύρος τιμών.

Αυτό μετακινεί το πρακτικό όριο απόφασης λίγο χαμηλότερα:

$$0{,}094 \text{ περίπου στη LDA} \qquad \rightarrow \qquad 0{,}090 \text{ περίπου στο Gaussian NB.}$$

Αυτή η μικρή αλλαγή αρκεί για να περάσουν περισσότερα πραγματικά red στην περιοχή πρόβλεψης `red`. Έτσι:

- το recall red ανεβαίνει από 0,2046 σε 0,2532,
- το F1 red ανεβαίνει από 0,3143 σε 0,3750,
- η accuracy ανεβαίνει από 0,7804 σε 0,7923.

Η AUC μένει σχεδόν ίδια, επειδή η σειρά των δειγμάτων ως προς τα `chlorides` είναι ήδη πολύ καλή και στις δύο μεθόδους. Η διαφορά είναι κυρίως στο τελικό σημείο απόφασης και στο πώς χρησιμοποιείται η διαφορετική διασπορά των δύο κλάσεων.

7.12 Συμπέρασμα για το υποερώτημα A(c)

Με βάση το Gaussian Naive Bayes:

1. Το `alcohol` δεν είναι κατάλληλη μοναδική μεταβλητή. Το μοντέλο προβλέπει όλα τα δείγματα ως `white`.
2. Το `pH` περιέχει κάποια πληροφορία, αλλά παραμένει μέτριο.
3. Τα `chlorides` είναι η καλύτερη μεταβλητή. Δίνουν την καλύτερη accuracy, balanced accuracy, recall red και F1 red.

Άρα, για το υποερώτημα A(c), η καλύτερη μοναδική προβλεπτική μεταβλητή είναι:

$$\boxed{\text{chlorides}}$$

Επιπλέον, ανάμεσα στις τρεις μεθόδους που εξετάσαμε μέχρι τώρα για τη μεταβλητή `chlorides`, το Gaussian NB έχει την καλύτερη τελική επίδοση στα test δεδομένα, κυρίως επειδή χειρίζεται καλύτερα τη διαφορετική διασπορά των red και white κρασιών.

8 Υποερώτημα A(ii): Καταλληλότερη προβλεπτική μεταβλητή

8.1 Τι ζητάει το A(ii)

Το A(ii) ζητά να απαντήσουμε ποια από τις τρεις μεταβλητές:

`alcohol`, `pH`, `chlorides`

είναι καταλληλότερη για την πρόβλεψη του τύπου κρασιού.

Για να απαντήσουμε σωστά, δεν πρέπει να κοιτάξουμε μόνο την accuracy. Όπως είδαμε, επειδή τα white κρασιά είναι πολύ περισσότερα, ένα μοντέλο μπορεί να έχει περίπου 75% accuracy απλώς επειδή προβλέπει σχεδόν πάντα `white`. Γι' αυτό κοιτάμε κυρίως:

- την **AUC**, δηλαδή τη διακριτική ικανότητα ανεξάρτητα από threshold,
- την **balanced accuracy**, επειδή λαμβάνει υπόψη και τις δύο κλάσεις,
- το **recall red**, επειδή η κλάση red είναι η δύσκολη μειοψηφική κλάση,
- και το **F1 red**, επειδή συνοψίζει την επίδοση στην κλάση red.

8.2 Συνολική εικόνα ανά μεταβλητή

Για κάθε μεταβλητή κρατάμε την καλύτερη επίδοση που πήραμε από τις τρεις μεθόδους:

Μεταβλητή	Μέθοδος	AUC	Bal. Acc.	F1 red
alcohol	Gaussian NB	0,5678	0,5000	0,0000
pH	Gaussian NB	0,6798	0,5366	0,1826
chlorides	Gaussian NB	0,9220 περίπου	0,6107	0,3750

Ο πίνακας δείχνει καθαρά ότι τα `chlorides` είναι η καλύτερη μεταβλητή. Η AUC τους είναι περίπου 0,92, πολύ υψηλότερη από το `pH` και το `alcohol`. Αυτό σημαίνει ότι τα `chlorides` βάζουν τα δείγματα σε πολύ καλύτερη σειρά από πιο πιθανό `white` προς λιγότερο πιθανό `white`.

Επιπλέον, τα `chlorides` έχουν και την καλύτερη balanced accuracy και το καλύτερο F1 red. Αυτό είναι σημαντικό, γιατί δείχνει ότι δεν βοηθούν μόνο να ταξινομήσουμε σωστά τα white, αλλά βοηθούν περισσότερο από τις άλλες μεταβλητές και στην αναγνώριση των red.

8.3 Γιατί τα chlorides είναι καλύτερα;

Η εξήγηση φαίνεται και από τα περιγραφικά χαρακτηριστικά:

$$\mu_{red} = 0{,}0855, \qquad \mu_{white} = 0{,}0462.$$

Τα κόκκινα κρασιά έχουν αρκετά υψηλότερη μέση τιμή `chlorides` από τα λευκά. Αντίθετα, στο `alcohol` οι μέσες τιμές είναι σχεδόν ίδιες:

$$10{,}4285 \text{ για red} \qquad 10{,}5535 \text{ για white.}$$

Άρα το `alcohol` δεν δίνει καθαρό σήμα. Το `pH` δίνει κάποιο σήμα, αλλά όχι τόσο ισχυρό όσο τα `chlorides`. Τα `chlorides` έχουν την καλύτερη σχέση ανάμεσα στη διαφορά των μέσων και στη διασπορά των τιμών.

8.4 Απάντηση στο A(ii)

Η καταλληλότερη προβλεπτική μεταβλητή για την πρόβλεψη του τύπου κρασιού είναι:

`chlorides`

Ο λόγος είναι ότι η μεταβλητή `chlorides` εμφανίζει την ισχυρότερη διαφοροποίηση ανάμεσα στα red και white κρασιά και δίνει την καλύτερη διακριτική ικανότητα σε όλες σχεδόν τις μετρικές που έχουν σημασία.

9 Υποερώτημα A(iii): Σύγκριση μεθόδων με μοναδική μεταβλητή τα Chlorides

9.1 Τι ζητάει το A(iii)

Το A(iii) ζητά να κρατήσουμε μόνο τη μεταβλητή `chlorides` και να συγκρίνουμε τις τρεις μεθόδους:

Logistic Regression, LDA, Gaussian NB.

Η ερώτηση είναι αν υπάρχει σημαντική διαφορά στην επίδοση των μεθόδων και ποια μέθοδος έχει την καλύτερη ακρίβεια.

9.2 Επίδοση στο test set

Στον παρακάτω πίνακα το `Rec.` σημαίνει recall. Άρα `Rec. white` είναι το ποσοστό των πραγματικών white που βρέθηκαν σωστά, ενώ `Rec. red` είναι το αντίστοιχο ποσοστό για τα red.

Μέθοδος	Acc.	Bal. Acc.	AUC	Rec. white	Rec. red	F1 red
Logistic Reg.	0,7558	0,5236	0,9220	0,9808	0,0665	0,1182
LDA	0,7804	0,5864	0,9220	0,9683	0,2046	0,3143
Gaussian NB	0,7923	0,6107	0,9214	0,9683	0,2532	0,3750

Με βάση το test set, η καλύτερη μέθοδος είναι το **Gaussian NB**. Έχει:

- τη μεγαλύτερη accuracy,
- τη μεγαλύτερη balanced accuracy,
- το μεγαλύτερο recall red,

- και το μεγαλύτερο F1 red.

Η AUC είναι σχεδόν ίδια και για τις τρεις μεθόδους, περίπου 0,92. Αυτό σημαίνει ότι και οι τρεις μέθοδοι βάζουν τα δείγματα σε παρόμοια καλή σειρά ως προς τα `chlorides`. Η μεγάλη διαφορά βρίσκεται στο τελικό όριο απόφασης, δηλαδή στο πότε κάθε μέθοδος αποφασίζει να πει `red`.

9.3 Confusion matrices με chlorides

Για να κρατήσουμε τον πίνακα μικρό, γράφουμε: `TN` = πραγματικό red που προβλέφθηκε ως red, `FP` = πραγματικό red που προβλέφθηκε ως white, `FN` = πραγματικό white που προβλέφθηκε ως red, και `TP` = πραγματικό white που προβλέφθηκε ως white.

Μέθοδος	TN	FP	FN	TP
Logistic Reg.	26	365	23	1175
LDA	80	311	38	1160
Gaussian NB	99	292	38	1160

Ο πίνακας δείχνει γιατί το Gaussian NB είναι καλύτερο:

- Η Logistic Regression βρίσκει μόνο 26 από τα 391 red.
- Η LDA βρίσκει 80 από τα 391 red.
- Το Gaussian NB βρίσκει 99 από τα 391 red.

Ταυτόχρονα, το Gaussian NB δεν κάνει περισσότερα λάθη στα white από την LDA: και τα δύο έχουν 38 white που ταξινομήθηκαν λανθασμένα ως red. Άρα το Gaussian NB κερδίζει κυρίως επειδή αναγνωρίζει περισσότερα red χωρίς να θυσιάζει επιπλέον white.

9.4 Υπάρχει σημαντική διαφορά;

Χρησιμοποιούμε τον έλεγχο **McNemar** για να δούμε αν η διαφορά είναι σημαντική. Ο McNemar είναι κατάλληλος εδώ, επειδή συγκρίνει δύο μοντέλα πάνω στα ίδια test δεδομένα. Με απλά λόγια, ελέγχει αν τα λάθη των δύο μεθόδων διαφέρουν ουσιαστικά ή αν η διαφορά θα μπορούσε να είναι απλή τύχη.

Σύγκριση	A σωστό/B λάθος	A λάθος/B σωστό	p-value
LogReg vs LDA	15	54	$2{,}61 \times 10^{-6}$
LogReg vs GNB	15	73	$2{,}56 \times 10^{-10}$
LDA vs GNB	0	19	$3{,}81 \times 10^{-6}$

Όλα τα p-values είναι πολύ μικρότερα από 0,05. Άρα οι διαφορές δεν φαίνεται να είναι τυχαίες. Υπάρχει **στατιστικά σημαντική διαφορά** στην επίδοση των μεθόδων.
Η σειρά επίδοσης είναι:

$$\text{Gaussian NB} > \text{LDA} > \text{Logistic Regression}.$$

9.5 Επιβεβαίωση με cross-validation

Για να μη βασιστούμε μόνο σε έναν συγκεκριμένο διαχωρισμό training/test, έγινε και 10-fold stratified cross-validation:

Μέθοδος	Mean Acc.	Mean Bal. Acc.	Mean AUC
LogReg	0,7535	0,5195	0,9184
LDA	0,7817	0,5859	0,9184
Gaussian NB	0,7915	0,6082	0,9176

Το cross-validation επιβεβαιώνει την ίδια εικόνα:

- το Gaussian NB έχει τη μεγαλύτερη μέση accuracy,
- έχει και τη μεγαλύτερη μέση balanced accuracy,
- ενώ η AUC παραμένει σχεδόν ίδια για όλες τις μεθόδους.

9.6 Απάντηση στο A(iii)

Ναι, υπάρχει σημαντική διαφορά στην επίδοση των μεθόδων όταν χρησιμοποιούμε μόνο τη μεταβλητή `chlorides`. Η καλύτερη μέθοδος είναι:

$$\boxed{\text{Gaussian Naive Bayes}}$$

Η καλύτερη επίδοσή του οφείλεται στο ότι εκμεταλλεύεται όχι μόνο τη διαφορά των μέσων τιμών ανάμεσα σε red και white, αλλά και τη διαφορετική διασπορά των δύο κλάσεων. Τα red έχουν πιο απλωμένες τιμές `chlorides`, ενώ τα white είναι πιο συγκεντρωμένα σε χαμηλές τιμές. Το Gaussian NB το λαμβάνει αυτό υπόψη και έτσι αναγνωρίζει περισσότερα red κρασιά από τις άλλες δύο μεθόδους.

10 Ερώτημα Β: Πρόβλεψη με όλες τις μεταβλητές

10.1 Τι ζητάει το Ερώτημα Β

Στο Ερώτημα Α χρησιμοποιούσαμε **μία μόνο προβλεπτική μεταβλητή κάθε φορά**. Αυτό έγινε επίτηδες για να δούμε πόση πληροφορία κρύβεται στο `alcohol`, στο `pH` και στα `chlorides` από μόνα τους.

Το Ερώτημα Β αλλάζει το πλαίσιο. Ζητάει να φτιάξουμε μοντέλα τα οποία χρησιμοποιούν **ταυτόχρονα όλες τις προβλεπτικές μεταβλητές**, εκτός από τη μεταβλητή `quality` την οποία δεν τη θεωρούμε χημικό χαρακτηριστικό. Πρακτικά αυτό σημαίνει ότι κάθε κρασί περιγράφεται από ένα διάνυσμα:

$$\mathbf{x} = (\text{fixed acidity},\ \text{volatile acidity},\ \text{citric acid},\ \text{residual sugar},\ \text{chlorides},$$

$$\text{free sulfur dioxide},\ \text{total sulfur dioxide},\ \text{density},\ \text{pH},\ \text{sulphates},\ \text{alcohol}).$$

Δηλαδή **11 χημικά χαρακτηριστικά** αντί για ένα. Ο στόχος παραμένει ο ίδιος: πρόβλεψη του `wine_type` ως `red` ή `white`.

Η εργασία ζητά να δοκιμάσουμε πέντε μεθόδους ταξινόμησης:

1. **K-Nearest Neighbors (KNN)**, με το K να επιλέγεται μέσω 5-fold cross-validation.
2. **Logistic Regression**.
3. **Linear Discriminant Analysis (LDA)**.
4. **Quadratic Discriminant Analysis (QDA)**.
5. **Gaussian Naive Bayes**.

Για κάθε μέθοδο πρέπει να υπολογιστεί:

- ο **confusion matrix** στα test δεδομένα,
- οι **δείκτες ακρίβειας** (accuracy, balanced accuracy, sensitivity, specificity, precision, F1-score),
- η **ROC καμπύλη** και η **AUC**.

Στο τέλος συγκρίνουμε τις μεθόδους και απαντάμε αν υπάρχει σημαντική διαφορά στην επίδοσή τους και ποια δίνει την καλύτερη τελική ακρίβεια.

10.2 Γιατί βοηθάει η χρήση πολλών μεταβλητών

Στο Ερώτημα Α είδαμε ότι ακόμα και η καλύτερη μεταβλητή, τα `chlorides`, δεν αρκούσε για να ξεχωρίσει τα κόκκινα και τα λευκά κρασιά με υψηλή ακρίβεια. Η καλύτερη accuracy που πήραμε με μία μόνο μεταβλητή ήταν γύρω στο 0,7923.

Όταν χρησιμοποιούμε πολλές μεταβλητές μαζί, το μοντέλο μπορεί να συνδυάσει διαφορετικές χημικές ενδείξεις. Για παράδειγμα:

- τα `chlorides` από μόνα τους ξεχωρίζουν καλά τα κόκκινα από τα λευκά,
- το `total sulfur dioxide` είναι πολύ υψηλότερο στα λευκά παρά στα κόκκινα,
- η `volatile acidity` είναι συνήθως μεγαλύτερη στα κόκκινα.

Όταν τα συνδυάσουμε, κερδίζουμε πληροφορία που καμία μεταβλητή δεν έχει μόνη της. Γι' αυτό στα αποτελέσματα που θα δούμε στη συνέχεια, η accuracy ανεβαίνει από το $\sim$0,79 πάνω από 0,94 σε όλες τις μεθόδους.

10.3 Γιατί δεν ενώνουμε απλά τις μεταβλητές χωρίς σκέψη

Όταν μια μέθοδος χρησιμοποιεί ταυτόχρονα πολλές μεταβλητές με διαφορετικές μονάδες και διαφορετικά εύρη τιμών, προκύπτει ένα πρακτικό πρόβλημα. Δείτε τι τιμές έχουν οι μεταβλητές μας:

- `chlorides`: τιμές γύρω στο 0,04–0,09,
- `alcohol`: τιμές γύρω στο 10–12,
- `total sulfur dioxide`: τιμές που ξεπερνούν τα 100.

Αν μια μέθοδος βασίζεται σε **αποστάσεις μεταξύ δειγμάτων**, όπως ο KNN, τότε μια μεταβλητή με μεγάλες τιμές (π.χ. total sulfur dioxide) θα κυριαρχεί πάνω σε μια μεταβλητή με μικρές τιμές (π.χ. chlorides) ακόμη και αν η δεύτερη έχει πολύ καλύτερη διακριτική ικανότητα. Για να μην συμβεί αυτό, κάνουμε **τυποποίηση** (standardization) πριν τον KNN:

$$x' = \frac{x - \mu}{\sigma}.$$

Μετά από αυτό, όλες οι μεταβλητές έχουν μέση τιμή 0 και τυπική απόκλιση 1, οπότε καμία δεν κυριαρχεί άδικα πάνω στις υπόλοιπες.

Το ίδιο σκεπτικό εφαρμόζεται και στη Logistic Regression: χωρίς τυποποίηση, οι βέλτιστοι συντελεστές β_j μπορεί να γίνουν πολύ μεγάλοι ή πολύ μικροί, και ο αλγόριθμος βελτιστοποίησης που υπολογίζει τους συντελεστές μπορεί να δυσκολευτεί να συγκλίνει. Η τυποποίηση κάνει τη βελτιστοποίηση πιο σταθερή και γρήγορη.

Για τις μεθόδους LDA, QDA και Gaussian NB η τυποποίηση δεν επηρεάζει το αποτέλεσμα, γιατί αυτές οι μέθοδοι "βλέπουν" τη δομή των κατανομών $P(\mathbf{x} \mid \text{κλάση})$ και όχι τις απόλυτες κλίμακες. Ωστόσο, η τυποποίηση είναι αβλαβής και την εφαρμόζουμε όπου χρειάζεται.

10.4 Τα δεδομένα για το Ερώτημα Β

Χρησιμοποιούμε τον ίδιο διαχωρισμό training/test με το Ερώτημα Α:

- 70% training, 30% test,
- `random_state = 6931` (ο ΑΕΜ μας),
- `stratify=y` ώστε να διατηρείται η αναλογία red/white και στις δύο υποδιαιρέσεις.

Η σύμβαση κωδικοποίησης παραμένει:

$$\text{red} = 0, \qquad \text{white} = 1.$$

11 Θεωρητικό υπόβαθρο για το Ερώτημα Β

Εδώ συνοψίζουμε τη θεωρία που χρειάζεται για τις νέες μεθόδους. Η Logistic Regression, η LDA και το Gaussian NB έχουν ήδη εξηγηθεί αναλυτικά στο Ερώτημα Α. Γι' αυτό εδώ εστιάζουμε στη γενίκευσή τους σε πολλές μεταβλητές, και προσθέτουμε τη θεωρία για δύο νέες μεθόδους: **KNN** και **QDA**.

11.1 Γενίκευση σε πολλές μεταβλητές

Στο Ερώτημα Α είχαμε πάντα ένα μόνο x. Στο Ερώτημα Β έχουμε ένα **διάνυσμα**:

$$\mathbf{x} = (x_1, x_2, \ldots, x_p)$$

όπου $p = 11$ στο πρόβλημά μας. Η γενική ιδέα κάθε μεθόδου παραμένει η ίδια, απλά γίνεται πολυδιάστατη:

- Η Logistic Regression γίνεται:

  $$P(\text{white} \mid \mathbf{x}) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x_1 + \cdots + \beta_p x_p)}}.$$

  Δηλαδή έχουμε έναν συντελεστή ανά μεταβλητή.

- Η LDA υποθέτει ότι κάθε κλάση ακολουθεί μια **πολυδιάστατη κανονική κατανομή** με δικό της κέντρο $\boldsymbol{\mu}_k$, αλλά **κοινό πίνακα συνδιακύμανσης** $\boldsymbol{\Sigma}$. Το αποτέλεσμα είναι **γραμμικά** όρια απόφασης στον πολυδιάστατο χώρο.

- Το Gaussian NB υποθέτει ότι κάθε μεταβλητή ξεχωριστά ακολουθεί κανονική κατανομή **μέσα στην κλάση της**, και ότι οι μεταβλητές είναι **ανεξάρτητες μέσα στην κλάση**. Αυτή είναι η "αφελής" (naive) υπόθεση. Πρακτικά σημαίνει:

  $$P(\mathbf{x} \mid \text{κλάση}) = \prod_{j=1}^{p} P(x_j \mid \text{κλάση}).$$

11.2 K-Nearest Neighbors (KNN)

Η μέθοδος KNN είναι πολύ διαφορετική από όλες τις προηγούμενες. Δεν φτιάχνει καμία εξίσωση και δεν μαθαίνει συντελεστές. Απλώς θυμάται όλα τα δείγματα του training set και, όταν έρθει ένα νέο κρασί για πρόβλεψη, κοιτάζει:

> Ποια είναι τα K πιο κοντινά κρασιά στο training set, και ποια κατηγορία (`red` ή `white`) είναι η πιο συχνή ανάμεσά τους;

11.2.1 Πώς μετράμε "κοντινό"

Για να πει ο KNN ποια κρασιά είναι κοντινά σε ένα νέο κρασί, μετράει **αποστάσεις** μεταξύ διανυσμάτων χαρακτηριστικών. Η συνηθέστερη απόσταση είναι η **Ευκλείδεια**:

$$d(\mathbf{x}, \mathbf{x}') = \sqrt{\sum_{j=1}^{p} (x_j - x'_j)^2}.$$

Αυτή είναι η απόσταση που θα υπολογίζαμε και στη γεωμετρία του σχολείου, απλώς τώρα δεν είναι σε 2 ή 3 διαστάσεις αλλά σε 11 διαστάσεις (όσες και οι μεταβλητές μας).

Το πρόβλημα που εξηγήσαμε πριν γίνεται καθαρό εδώ. Αν το `total sulfur dioxide` παίρνει τιμές από 10 έως 300 και τα `chlorides` από 0,02 έως 0,3, η διαφορά στα chlorides είναι πάντα μικρή σε απόλυτους αριθμούς. Έτσι, η Ευκλείδεια απόσταση θα κυριαρχείται σχεδόν πάντα από το `total sulfur dioxide` και ο KNN δεν θα "βλέπει" τα chlorides. Γι' αυτό **τυποποιούμε** όλες τις μεταβλητές πριν εφαρμόσουμε KNN.

11.2.2 Πώς ψηφίζουν οι γείτονες

Έστω ότι έχουμε έναν νέο πίνακα τιμών $\mathbf{x}^*$. Ο KNN κάνει τα εξής:

1. Υπολογίζει την απόσταση του $\mathbf{x}^*$ από κάθε δείγμα του training set.
2. Επιλέγει τα K πιο κοντινά.
3. Κοιτάζει τις κλάσεις τους και προβλέπει την κλάση με περισσότερες ψήφους (πλειοψηφία).

Επίσης, ο KNN μπορεί να δώσει "πιθανότητα". Αν από τους $K = 9$ γείτονες οι 7 είναι white και οι 2 είναι red, τότε:

$$P(\text{white} \mid \mathbf{x}^*) = \frac{7}{9} \approx 0{,}778.$$

Αυτή η πιθανότητα είναι χρήσιμη για τον υπολογισμό της AUC.

11.2.3 Γιατί το K είναι σημαντικό

Το K είναι η πιο σημαντική παράμετρος του KNN:

- Μικρό K (π.χ. $K = 1$): το μοντέλο "βλέπει" μόνο τον πιο κοντινό γείτονα. Είναι πολύ ευαίσθητο σε θόρυβο. Ένα μεμονωμένο κρασί που "μοιάζει" με την άλλη κλάση μπορεί να προκαλέσει λάθος. Αυτό λέγεται **overfitting**.
- Μεγάλο K (π.χ. $K = 100$): το μοντέλο ψηφίζει από πολύ μεγάλη γειτονιά. Οι προβλέψεις γίνονται "μέσες" και χάνει το μοντέλο τοπική λεπτομέρεια. Αυτό λέγεται **underfitting**.

Επομένως, θέλουμε ένα K που να βρίσκει τη χρυσή τομή. Για να τη βρούμε, η εκφώνηση λέει να χρησιμοποιήσουμε **5-fold cross-validation**.

11.2.4 Τι είναι το 5-fold cross-validation

Η διασταυρούμενη επικύρωση (cross-validation) είναι ένας τρόπος να δοκιμάζουμε διαφορετικές τιμές ενός μοντέλου χωρίς να αγγίζουμε το test set. Στο 5-fold cross-validation:

1. Χωρίζουμε το *training set* σε 5 ισομεγέθη κομμάτια (folds).
2. Για κάθε τιμή K που θέλουμε να δοκιμάσουμε:
   (α′) Εκπαιδεύουμε τον KNN σε 4 από τα 5 folds.
   (β′) Αξιολογούμε στο 5ο fold.
   (γ′) Επαναλαμβάνουμε ώστε κάθε fold να παίξει μία φορά τον ρόλο του "μικρού test set".
   (δ′) Παίρνουμε τη μέση τιμή της accuracy.
3. Επιλέγουμε το K με τη μεγαλύτερη μέση CV accuracy.

Το καλό με αυτή τη διαδικασία είναι ότι η επιλογή του K γίνεται **χωρίς να δούμε ούτε ένα δείγμα από το test set**. Έτσι τα αποτελέσματα στο test set παραμένουν αμερόληπτα.

11.3 Quadratic Discriminant Analysis (QDA)

Η QDA είναι η "αδελφή" της LDA. Και οι δύο υποθέτουν ότι κάθε κλάση περιγράφεται από μια **πολυδιάστατη κανονική κατανομή**. Η διαφορά τους είναι μία και πολύ σημαντική:

- **LDA**: υποθέτει ότι οι **συνδιακυμάνσεις** των δύο κλάσεων είναι ίδιες. Δηλαδή οι κατανομές των red και των white "έχουν το ίδιο σχήμα", απλώς κάθονται σε διαφορετικά κέντρα.
- **QDA**: επιτρέπει σε κάθε κλάση να έχει **τον δικό της πίνακα συνδιακύμανσης**. Δηλαδή επιτρέπει και διαφορετικά σχήματα κατανομών.

Αυτή η ελευθερία έχει συνέπεια: το όριο απόφασης δεν είναι πια ευθεία γραμμή (ή υπερ-επίπεδο σε πολλές διαστάσεις), αλλά **καμπύλη δευτέρου βαθμού**. Γι' αυτό λέγεται *Quadratic*.

11.3.1 Πότε κερδίζει η QDA

Η QDA είναι πιο ευέλικτη από τη LDA. Άρα, όταν οι δύο κλάσεις **όντως έχουν διαφορετικές διασπορές**, η QDA μπορεί να προσφέρει καλύτερη προσαρμογή. Στο Ερώτημα Α είδαμε π.χ. ότι τα `chlorides` των red ήταν πολύ πιο απλωμένα από αυτά των white. Αυτό είναι ακριβώς το σενάριο όπου η QDA μπορεί να βοηθήσει.

11.3.2 Πότε χάνει η QDA

Όμως η QDA έχει πολλές παραμέτρους να εκτιμήσει: έναν ολόκληρο πίνακα συνδιακύμανσης ανά κλάση. Αν τα δεδομένα μιας κλάσης είναι λίγα, ή αν πολλές μεταβλητές είναι περίπου γραμμικά εξαρτημένες μεταξύ τους, τότε οι εκτιμήσεις γίνονται ασταθείς. Εδώ η LDA, που έχει λιγότερες παραμέτρους, τα καταφέρνει καλύτερα.

Αυτή τη διαφορά θα τη δούμε και στα πραγματικά μας αποτελέσματα: η QDA θα πετύχει χειρότερη accuracy από τη LDA.

11.4 Τι δείχνει η ROC καμπύλη και η AUC σε αυτό το πλαίσιο

Η ROC και η AUC έχουν εξηγηθεί αναλυτικά στο Ερώτημα Α. Εδώ μόνο υπενθυμίζουμε τη βασική ιδέα:

- Κάθε μοντέλο δίνει, εκτός από την τελική πρόβλεψη, και **πιθανότητες**. Για κάθε κρασί δίνει ένα $P(\text{white} \mid \mathbf{x})$.
- Η **ROC καμπύλη** δείχνει τι συμβαίνει αν δοκιμάσουμε όλα τα δυνατά κατώφλια απόφασης. Ο x-άξονας είναι το False Positive Rate και ο y-άξονας το True Positive Rate.
- Η **AUC** (Area Under Curve) είναι ένας αριθμός από 0,5 έως 1 που συνοψίζει πόσο καλά το μοντέλο βάζει σε σειρά τα κρασιά. Κοντά στο 1 σημαίνει εξαιρετική διάκριση. Κοντά στο 0,5 σημαίνει σχεδόν τυχαίο.

12 Πώς λύσαμε το Ερώτημα Β

12.1 Δομή του κώδικα

Ο κώδικας βρίσκεται στο αρχείο `Q2_all_predictors.py`. Η δομή του είναι:

1. Φορτώνουμε τα δεδομένα και ορίζουμε τις προβλεπτικές μεταβλητές (όλες πλην `quality`).
2. Κωδικοποιούμε την κλάση: red $= 0$, white $= 1$.
3. Χωρίζουμε σε training/test με `random_state=6931` και `stratify=y`.
4. Επιλέγουμε το K του KNN μέσω 5-fold stratified CV στο training set.
5. Ορίζουμε τα υπόλοιπα τέσσερα μοντέλα.
6. Εκπαιδεύουμε το καθένα στο training set.
7. Υπολογίζουμε confusion matrix, accuracy, balanced accuracy, AUC κ.ά. στο test set.
8. Σχεδιάζουμε τις ROC καμπύλες όλων των μεθόδων μαζί.
9. Κάνουμε paired **McNemar tests** για να δούμε αν οι διαφορές μεταξύ των μεθόδων είναι στατιστικά σημαντικές.

12.2 Τυποποίηση μέσω Pipeline

Για τις μεθόδους που χρειάζονται τυποποίηση (KNN και Logistic Regression), χρησιμοποιούμε `sklearn.pipeline.Pipeline`:

```
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.neighbors import KNeighborsClassifier

knn_pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("knn", KNeighborsClassifier()),
])
```

Το Pipeline εξασφαλίζει ότι η τυποποίηση γίνεται **μόνο πάνω στο training set** και μετά εφαρμόζεται *με τις ίδιες παραμέτρους* στο test set. Αν κάναμε τυποποίηση σε όλα τα δεδομένα μαζί, θα είχαμε **data leakage**: το test set θα επηρέαζε πληροφορίες που παίρνει το μοντέλο στην εκπαίδευση.

12.3 Επιλογή του K με 5-fold CV

Για την αναζήτηση του βέλτιστου K χρησιμοποιούμε το `GridSearchCV`:

```
from sklearn.model_selection import StratifiedKFold, GridSearchCV

k_grid = list(range(1, 31))
cv5 = StratifiedKFold(
    n_splits=5, shuffle=True, random_state=6931
)
gs = GridSearchCV(
    knn_pipe,
    {"knn__n_neighbors": k_grid},
    scoring="accuracy",
    cv=cv5,
    n_jobs=-1,
)
gs.fit(X_train, y_train)
best_k = gs.best_params_["knn__n_neighbors"]
```

Το `StratifiedKFold` φροντίζει ώστε κάθε fold να έχει περίπου την ίδια αναλογία red/white όπως ο αρχικός πληθυσμός. Αυτό είναι σημαντικό σε ανισόρροπα σύνολα όπως το δικό μας.

13 Επιλογή του K με 5-fold CV

13.1 Τι βρήκαμε

Η 5-fold stratified CV στο training set έδωσε τα παρακάτω (επιλεγμένα) αποτελέσματα:

K	Mean CV Accuracy	Std.
1	0,9447	0,0031
3	0,9665	0,0020
5	0,9679	0,0016
7	0,9679	0,0022
9	**0,9692**	**0,0013**
11	0,9687	0,0018
15	0,9676	0,0019
21	0,9684	0,0020
30	0,9668	0,0020

Το βέλτιστο K είναι:

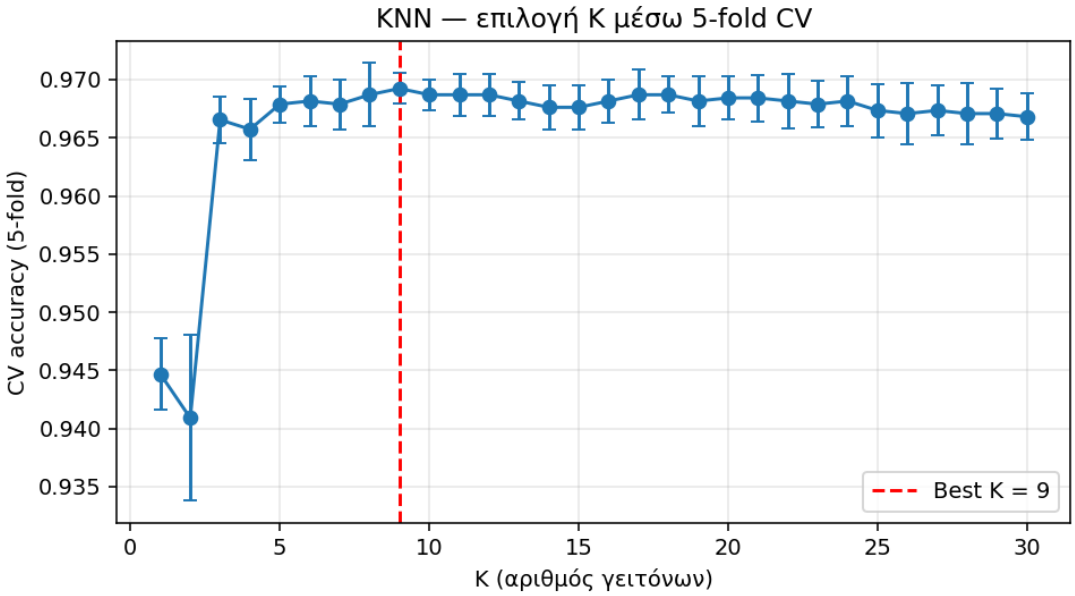
$$\boxed{K = 9}$$

με μέση CV accuracy $0{,}9692$.

13.2 Γιατί αυτό το K έχει νόημα

Βλέπουμε μια χαρακτηριστική εικόνα:

- Για $K = 1$, η accuracy είναι χαμηλότερη ($\approx 0{,}9447$). Αυτό οφείλεται στο ότι ένας μόνο γείτονας είναι πολύ ευαίσθητος σε "περίεργα" κρασιά.
- Για K μεταξύ 5 και 15 η accuracy είναι σχεδόν η ίδια και κοντά στο μέγιστο. Ο $K = 9$ κερδίζει οριακά.
- Για πολύ μεγάλα K (π.χ. 25–30) η accuracy αρχίζει πάλι να μειώνεται, επειδή το μοντέλο "εξομαλύνει" πάρα πολύ.



Σχήμα 1: Επιλογή του K για KNN μέσω 5-fold stratified cross-validation. Η κόκκινη διακεκομμένη γραμμή δείχνει το επιλεγμένο $K = 9$.

14 Αποτελέσματα στο test set

14.1 Συνοπτικός πίνακας

Μετά την επιλογή $K = 9$, εκπαιδεύσαμε όλες τις μεθόδους στο training set και τις αξιολογήσαμε στο test set (30% των δεδομένων). Ο πίνακας που ακολουθεί συνοψίζει τα κυριότερα μέτρα ακρίβειας:

Μέθοδος	Accuracy	Bal. Acc.	AUC	F1 white	F1 red
KNN ($K = 9$)	**0,9704**	**0,9614**	0,9664	**0,9804**	**0,9401**
Logistic Reg.	0,9654	0,9529	**0,9692**	0,9771	0,9296
LDA	0,9679	0,9546	0,9678	0,9788	0,9344
QDA	0,9559	0,9415	0,9605	0,9708	0,9107
Gaussian NB	0,9440	0,9344	0,9575	0,9625	0,8894

14.2 Σύντομη υπενθύμιση των metrics

Επειδή οι αριθμοί είναι κοντά μεταξύ τους, αξίζει να ξαναθυμηθούμε τι δείχνει ο καθένας πριν τους ερμηνεύσουμε:

- **Accuracy**: το συνολικό ποσοστό σωστών προβλέψεων. Εδώ όλες οι μέθοδοι ξεπερνούν το 0,94.
- **Balanced accuracy**: ο μέσος όρος των recall για τις δύο κλάσεις (recall red και recall white). Προστατεύει από το να "κρύβεται" μια αδυναμία στην πιο σπάνια κλάση.
- **AUC**: πόσο καλά το μοντέλο βάζει σε σειρά τα κρασιά, ανεξάρτητα από το threshold. Όλες οι μέθοδοι εδώ ξεπερνούν το 0,95.
- **F1 white**: αρμονικός μέσος precision/recall για τα white.
- **F1 red**: αρμονικός μέσος precision/recall για τα red. Είναι πιο ενημερωτικό, γιατί red είναι η πιο σπάνια κλάση.

14.3 Ερμηνεία των αποτελεσμάτων

Από τον πίνακα προκύπτουν τρία καθαρά μηνύματα:

1. **Και οι πέντε μέθοδοι λειτουργούν πολύ καλά**. Η accuracy είναι πάνω από 94% σε όλες τις περιπτώσεις. Αυτό δείχνει ότι, όταν χρησιμοποιούμε όλες τις χημικές μεταβλητές μαζί, το πρόβλημα διάκρισης red από white γίνεται σχετικά εύκολο.
2. **Ο KNN κερδίζει οριακά**. Έχει την καλύτερη accuracy (0,9704), την καλύτερη balanced accuracy (0,9614) και το καλύτερο F1 για τα red. Αυτό δεν είναι τυχαίο: ο KNN δεν υποθέτει καμία κατανομή ή γραμμικό όριο. Αξιοποιεί "τοπικά" μοτίβα στα δεδομένα και έτσι μπορεί να πιάσει λεπτότερες διαφορές που οι άλλες μέθοδοι χάνουν λόγω των υποθέσεών τους.
3. **Το Gaussian NB πέφτει τελευταίο**. Αυτό είναι αναμενόμενο: το Gaussian NB υποθέτει ότι οι μεταβλητές είναι *ανεξάρτητες μέσα σε κάθε κλάση*. Στο πρόβλημά μας αυτό δεν ισχύει. Για παράδειγμα, το `total sulfur dioxide` και το `free sulfur dioxide` είναι ισχυρά συσχετισμένα. Όταν το NB αγνοεί τη συσχέτιση, υπερεκτιμά την πληροφορία που δίνει το δείγμα.

14.4 Γιατί η QDA είναι χειρότερη από τη LDA εδώ

Ένα σημείο που αξίζει προσοχή: η QDA είναι πιο ευέλικτη από τη LDA, αλλά στο πρόβλημά μας είναι πιο ανακριβής (0,9559 έναντι 0,9679). Γιατί;

Ο λόγος είναι ότι στις 11 διαστάσεις η QDA πρέπει να εκτιμήσει έναν πλήρη πίνακα συνδιακύμανσης $\mathbf{\Sigma}_k$ για την κάθε κλάση. Αυτό σημαίνει πολλές παραμέτρους (της τάξης του $p^2/2$ για κάθε κλάση). Όταν υπάρχουν μεταβλητές με ισχυρές συσχετίσεις (όπως οι χημικοί δείκτες του κρασιού), αυτές οι εκτιμήσεις γίνονται ασταθείς.

Επιπλέον, στα δεδομένα μας η *σχετική δομή συσχέτισης* των μεταβλητών δεν είναι τόσο διαφορετική ανάμεσα στις δύο κλάσεις ώστε να αξίζει ο QDA την επιπλέον πολυπλοκότητα. Η LDA, με ένα κοινό $\mathbf{\Sigma}$, πετυχαίνει ένα καλύτερο συμβιβασμό μεταξύ "ευελιξίας" και "στατιστικής σταθερότητας".

14.5 Confusion matrices

Για να δούμε ποιες προβλέψεις λείπουν από κάθε μοντέλο, παραθέτουμε τους confusion matrices στο test set. Όπως και πριν: `TN` = πραγματικό red ταξινομημένο ως red, `FP` = πραγματικό red ταξινομημένο ως white, `FN` = πραγματικό white ταξινομημένο ως red, `TP` = πραγματικό white ταξινομημένο ως white.

Μέθοδος	TN	FP	FN	TP
KNN ($K = 9$)	369	22	25	1173
Logistic Reg.	363	28	27	1171
LDA	363	28	23	1175
QDA	357	34	36	1162
Gaussian NB	358	33	56	1142

Τα συνολικά κρασιά στο test set είναι 1589, εκ των οποίων 391 είναι red και 1198 είναι white (αναλογία $\approx 75{:}25$, ίδια με την συνολική).

Ο KNN κάνει σχετικά ισορροπημένα λάθη. Βρίσκει:

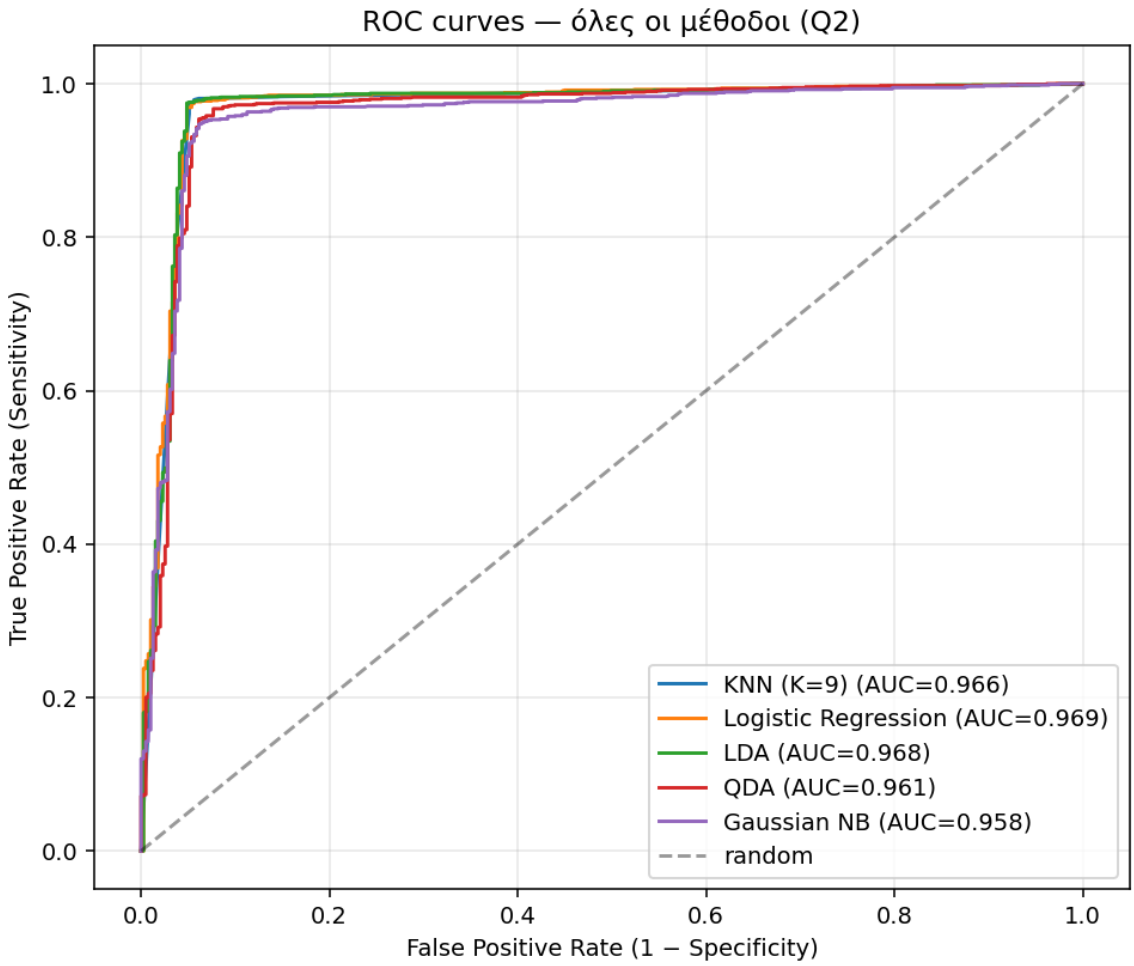
- 369 από τα 391 red, δηλαδή recall red $\approx 0{,}944$,
- 1173 από τα 1198 white, δηλαδή recall white $\approx 0{,}979$.

Αντίθετα, το Gaussian NB κάνει σαφώς περισσότερα λάθη στα white (56 λάθη), παρόλο που έχει αριθμό red-σωστών παρόμοιο με τις άλλες μεθόδους. Αυτό ρίχνει την accuracy του.

15 ROC καμπύλες και AUC

15.1 Η εικόνα

Σχεδιάσαμε όλες τις ROC καμπύλες μαζί στον ίδιο άξονα για εύκολη σύγκριση.



Σχήμα 2: ROC καμπύλες των πέντε μεθόδων του Ερωτήματος Β. Η διαγώνια γραμμή αντιστοιχεί σε ένα μοντέλο που κάνει τυχαίες προβλέψεις.

Παρατηρούμε ότι και οι πέντε καμπύλες είναι αρκετά κοντά στην πάνω αριστερή γωνία του διαγράμματος. Καμία δεν πλησιάζει τη διαγώνιο, άρα κανένα μοντέλο δεν είναι "τυχαίο". Οι τιμές AUC είναι:

Μέθοδος	AUC
Logistic Regression	**0,9692**
LDA	0,9678
KNN ($K = 9$)	0,9664
QDA	0,9605
Gaussian NB	0,9575

15.2 Γιατί η AUC της Logistic Regression είναι η μεγαλύτερη, αλλά όχι η accuracy της

Σε αυτό το σημείο υπάρχει ένα ενδιαφέρον εύρημα. Η Logistic Regression έχει τη *μεγαλύτερη* AUC (0,9692), όμως ο KNN έχει τη *μεγαλύτερη* accuracy στο test set (0,9704). Δεν είναι αντίφαση.

Θυμίζουμε ότι:

- Η **AUC** μετράει πόσο καλά το μοντέλο βάζει τα δείγματα σε σειρά από "πιο πιθανό white" προς "λιγότερο πιθανό white". Άρα αντιστοιχεί στην ποιότητα των πιθανοτήτων, ανεξάρτητα από το threshold.

- Η **accuracy** αφορά την τελική ετικέτα, δηλαδή εξαρτάται από το συγκεκριμένο threshold (εδώ το default 0,5).

Η Logistic Regression δίνει πολύ καθαρές, καλά βαθμονομημένες πιθανότητες, οπότε η σειρά των δειγμάτων είναι εξαιρετική (υψηλή AUC). Ο KNN δεν έχει τόσο "ομαλές" πιθανότητες (αφού κάθε πιθανότητα προκύπτει ως κλάσμα k/9), αλλά στο default threshold οι τελικές του προβλέψεις τυχαίνει να είναι ελαφρώς καλύτερες στο συγκεκριμένο test set.

Η ουσία είναι ότι οι δύο μετρικές μετράνε διαφορετικά πράγματα και γι' αυτό δεν χρειάζεται να συμφωνούν πάντα.

16 Είναι στατιστικά σημαντικές οι διαφορές;

16.1 McNemar test

Όπως και στο Ερώτημα Α, για να ελέγξουμε αν οι διαφορές ανάμεσα στις μεθόδους είναι ουσιαστικές ή αν θα μπορούσαν να προκύψουν από τύχη, χρησιμοποιούμε τον **McNemar test**. Αυτός συγκρίνει δύο μοντέλα πάνω στα *ίδια test δεδομένα* και κοιτάζει τα δείγματα όπου το ένα μοντέλο πέτυχε και το άλλο απέτυχε.

Σύγκριση	A σωστό/B λάθος	A λάθος/B σωστό	p-value
KNN vs LogReg	13	5	0,0963
KNN vs LDA	9	5	0,4240
KNN vs QDA	27	4	$7{,}77 \times 10^{-5}$
KNN vs Gaussian NB	44	2	$1{,}49 \times 10^{-9}$
LogReg vs LDA	2	6	0,2891
LogReg vs QDA	21	6	0,0071
LogReg vs Gaussian NB	38	4	$3{,}54 \times 10^{-7}$
LDA vs QDA	24	5	0,0008
LDA vs Gaussian NB	42	4	$4{,}89 \times 10^{-8}$
QDA vs Gaussian NB	25	6	0,0012

16.2 Τι μας λένε τα p-values

Με κατώφλι σημαντικότητας $\alpha = 0{,}05$:

- **KNN, Logistic Regression και LDA είναι μεταξύ τους πρακτικά ισοδύναμοι**. Τα p-values στα ζευγάρια KNN-LogReg (0,0963), KNN-LDA (0,4240) και LogReg-LDA (0,2891) είναι όλα $> \alpha$. Δηλαδή οι τρεις κορυφαίοι ταξινομητές δεν μπορούν να ξεχωριστούν στατιστικά σημαντικά σε αυτό το test set.

- **Η QDA είναι χειρότερη από τις τρεις κορυφαίες**. Οι συγκρίσεις LogReg vs QDA (p=0,0071) και LDA vs QDA (p=0,0008) είναι σημαντικές. Άρα η πτώση στην accuracy της QDA δεν είναι τυχαία.

- **Το Gaussian NB είναι σαφώς χειρότερο από όλες τις άλλες**. Όλα τα p-values που περιλαμβάνουν Gaussian NB είναι πολύ μικρότερα του 0,05 (το μεγαλύτερο είναι $3{,}54 \times 10^{-7}$). Άρα η υστέρηση του NB είναι στατιστικά ισχυρή.

Συνολικά η σειρά επίδοσης που προκύπτει είναι:

$$\{\text{KNN, LogReg, LDA}\} \ > \ \text{QDA} \ > \ \text{Gaussian NB}.$$

Μέσα στο πρώτο σύνολο οι διαφορές είναι μικρές και μη σημαντικές.

17 Τελικά συμπεράσματα για το Ερώτημα Β

17.1 Υπάρχει σημαντική διαφορά στην επίδοση των μεθόδων;

Ναι, αλλά όχι παντού.
Υπάρχουν τρεις κατηγορίες:

- Πρώτη ομάδα: **KNN ($K = 9$), Logistic Regression, LDA**. Και οι τρεις πετυχαίνουν accuracy γύρω στο 0,965–0,970 και AUC γύρω στο 0,967–0,969. Ο McNemar test δεν βρίσκει στατιστικά σημαντικές διαφορές μεταξύ τους.
- Δεύτερη ομάδα: **QDA**. Έχει μικρότερη accuracy (0,9559) και η διαφορά της από την πρώτη ομάδα είναι στατιστικά σημαντική.
- Τρίτη ομάδα: **Gaussian Naive Bayes**. Έχει τη χαμηλότερη accuracy (0,9440), και η διαφορά του από όλες τις άλλες μεθόδους είναι στατιστικά σημαντική.

17.2 Ποια μέθοδος έχει την καλύτερη ακρίβεια;

Με βάση την accuracy στο test set, η καλύτερη μέθοδος είναι:

$$\boxed{\text{KNN με } K = 9}$$

με accuracy $0{,}9704$ και balanced accuracy $0{,}9614$.

Ωστόσο, με βάση την AUC, η καλύτερη μέθοδος είναι η **Logistic Regression** (AUC $0{,}9692$). Επειδή όμως οι διαφορές μέσα στην κορυφαία ομάδα (KNN, LogReg, LDA) *δεν είναι στατιστικά σημαντικές*, μπορούμε να πούμε ότι και οι τρεις αυτές μέθοδοι είναι υποψήφιες "νικήτριες".

Αν έπρεπε να επιλέξουμε μία μόνο μέθοδο για πρακτική χρήση, τα κριτήρια είναι:

- Αν θέλουμε το απλούστερο και πιο ερμηνεύσιμο μοντέλο, διαλέγουμε **Logistic Regression** ή **LDA**, γιατί δίνουν επίσης συντελεστές που εξηγούν τη συνεισφορά κάθε μεταβλητής.
- Αν θέλουμε την υψηλότερη δυνατή τελική accuracy σε αυτό το συγκεκριμένο test set, διαλέγουμε **KNN ($K = 9$)**.

17.3 Γιατί το Ερώτημα Β δίνει πολύ καλύτερα αποτελέσματα από το Ερώτημα Α

Στο Ερώτημα Α η καλύτερη accuracy ήταν $0{,}7923$ (Gaussian NB με `chlorides`). Στο Ερώτημα Β η χειρότερη accuracy είναι $0{,}9440$ (Gaussian NB με όλες τις μεταβλητές).

Αυτή η τεράστια βελτίωση προέρχεται κυρίως από δύο παράγοντες:

1. **Συνδυασμός πληροφορίας**. Μεταβλητές όπως το `total sulfur dioxide`, η `volatile acidity` και η `density` έχουν η καθεμία πληροφορία που μόνη της δεν είχαμε κοιτάξει. Όταν το μοντέλο τις βλέπει όλες μαζί, μπορεί να συνδυάσει ενδείξεις και να πάρει πολύ πιο ασφαλείς αποφάσεις.
2. **Οριακές περιπτώσεις γίνονται ξεκάθαρες**. Στο Ερώτημα Α, ένα κρασί με "ουδέτερη" τιμή chlorides ήταν δύσκολο να ταξινομηθεί. Στο Ερώτημα Β, τέτοιες οριακές περιπτώσεις "ξεκλειδώνονται" γιατί κάποια άλλη μεταβλητή (π.χ. η πυκνότητα ή το sulfur dioxide) συχνά δίνει καθαρό σήμα.

17.4 Σύνοψη

Η απάντηση στο Ερώτημα Β μπορεί να συνοψιστεί σε δύο γραμμές:

- Και οι πέντε μέθοδοι λύνουν το πρόβλημα πολύ καλά όταν έχουν όλες τις χημικές μεταβλητές.
- Υπάρχει στατιστικά σημαντική διαφορά ανάμεσα στις τρεις κορυφαίες (KNN, LogReg, LDA) και τις δύο τελευταίες (QDA, Gaussian NB). Η **υψηλότερη τελική accuracy ανήκει στον KNN με** $K = 9$, ενώ η **υψηλότερη AUC ανήκει στη Logistic Regression**.